arXiv:2605.09650v1 [cs.AI] 10 May 2026

Workspace Optimization: How to Train Your Agent

Elad Sarafian[*]
NVIDIA
esarafian@nvidia.com

Gal Kaplun[*]
NVIDIA
gkaplun@nvidia.com

Ron Banner
NVIDIA
rbanner@nvidia.com

Daniel Soudry
NVIDIA & Technion
daniel.soudry@gmail.com

Boris Ginsburg
NVIDIA
bginsburg@nvidia.com

Abstract

Modern agents built on frontier language models often cannot adapt their weights. What, then, remains trainable? We argue it is the agent's *workspace*, the structured external substrate it reads, writes, and tests; we call its evolution *workspace optimization*. Workspace optimization targets hard multi-turn environments where a frontier model has strong priors but cannot solve the task in a single shot, so the agent must learn through interaction. We propose a principled way to evolve the workspace, mirroring the structure of weight-space training: artifacts in place of parameters, evidence in place of data, counterexamples in place of losses, and textual feedback in place of gradients. We instantiate the idea in DREAMTEAM, a multi-agent harness for ARC-AGI-3 whose roles build an executable world model, plan, hypothesize, probe, strategize, and route failures. On the current 25-game ARC-AGI-3 public set under the official scoring protocol and averaged over two independent runs, DREAMTEAM improves the SOTA protocol-matched agent's score from 36% to 38.4%, while using 31% fewer environment actions per game. See ARC Prize scorecard record (single run): https://arcprize.org/scorecards/831c83cf-b969-45fc-a6ce-27f9b3c4105c.

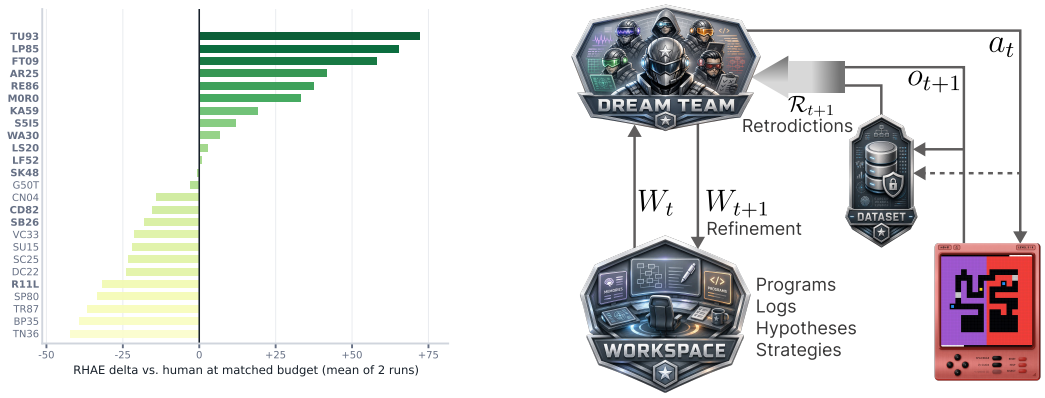
1 Introduction

Machine learning usually explains adaptation as movement in weight space. Data produces a loss, the loss produces an update, and enough updates change the model. That recipe shapes the default story for agentic learning: when the system fails, collect experience and train the model.

Many deployed agents [Anthropic, 2025, OpenAI, 2025] cannot follow that story. Fine-tuning is too slow for an online loop in which the agent must adapt within a single episode, and in many settings weight access is not on the table to begin with: the model sits behind an API, or the traces a deployed agent generates contain content that should not be baked into a checkpoint. The practical situation is that the model is fixed and the agent still has to learn. What, then, is the trainable state?

Imagine dropping a person into an unfamiliar video game with no instructions. The buttons do something, the screen changes in response, and some of the objects on screen turn out to matter while others are decoration. The goal is not stated (this is the setting of the ARC-AGI-3 benchmark [ARC Prize Foundation, 2026a]). Most people figure things out anyway by reasoning from prior similar experiences: they press buttons, watch what changes, form a rough theory, and start playing on purpose. The task, the environment, and a workable plan all come out of the same interaction.

[*]Equal contribution.



(a) (b)

Figure 1: **(a)** Per-game RHAE gap $\Delta_g = \mathrm{RHAE}_{\textsc{DreamTeam}}(g) - \mathrm{RHAE}_{\mathrm{human}}(g)$, averaged over our two independent runs of DREAMTEAM, with the human score sampled at DREAMTEAM's action budget; bold game labels mark equal-level finishers. **(b)** The DREAMTEAM workspace-optimization loop: retrodiction over historical evidence yields an updated workspace (hypotheses, strategies, logs) that roles read on the next step to refine their decisions.

In standard formulations, the input specifies the problem. In such an unfamiliar setting, the specification is hidden inside the environment instead. The question is not only *what action should the agent take?* It is *what should the agent learn next, where should it store that learning, and how should it change later behavior?* For example, two agents reading the same history can still end up in different places. Consider a door that opens when the player steps on a blue tile. One agent treats the surprise as a counterexample, attributes the failure to its transition rule, and patches the rule. The other appends a note for later retrieval. Both share the same frozen weights; they differ in the state they keep around the model.

We call this state the *workspace*, after the global-workspace framing in cognitive science [Baars, 2005]. It is the structured material the agent reads, writes, and tests around its calls to the model: some pieces are durable across the episode, some pieces are local to a single step (the parse the agent just did, the probe it is considering). All of it is editable. A workspace is not merely memory; it is the machinery that decides which observations turn into state, which predictions get logged as commitments, and which surprises change future behavior.

This paper studies *workspace optimization*: adapting a frozen-model agent by editing the workspace instead of the weights. The agent acts through the workspace, compares what it expected with what happened, and changes the state that later calls will read. The interesting question is credit assignment: when something goes wrong, what gets fixed? Different failures point at different parts of the workspace: misreading what is happening and mispredicting what will happen call for different fixes.

We instantiate this idea in DREAMTEAM, a multi-agent orchestrator for unfamiliar interactive games. Its workspace contains executable artifacts for observation, dynamics, and strategy, which the agents read, edit, and test around each action step. Each action is treated as a test of the workspace's predictions: a confirmed prediction is supporting evidence for the responsible artifacts, and a contradicted one is a counterexample routed to the artifact most responsible for the failure. The proposed repair is then replayed against earlier evidence to catch regressions before the next step. Early in a game, the loop favors probes that reduce uncertainty; as World-Model (WM) checks improve, control shifts toward model-based planning and efficient execution against the WM.

Contributions. This paper makes three contributions.

- **Workspace optimization.** We formulate workspace optimization, a framework in which frozen-model agents adapt by editing a structured workspace of typed artifacts. Each surprise is routed to the artifact responsible, recasting credit assignment as which artifact to edit rather than how much to update a weight (Section 2).
- **DREAMTEAM.** We instantiate workspace optimization as DREAMTEAM, a DreamerV3-inspired [Hafner et al., 2023] multi-agent harness whose roles own observation, dynamics, strategy,

probing, critique, and arbitration, each acting through and updating its part of the workspace from contradicted predictions (Section 4).
- **Efficient new state of the art.** On the 25-game ARC-AGI-3 public set and averaged over two independent runs, DREAMTEAM reaches 38.4% RHAE (Relative Human Action Efficiency, the official ARC-AGI-3 score; full definition in Section 5), surpassing the prior protocol-matched single-run baseline of 36%, while using 31% fewer environment actions per game (mean of 444 vs the baseline's 643) (Figure 1, Section 5).

Related work. Model-based RL and neural World-Models learn predictive state and dynamics in parameter space and plan with imagined rollouts [Ha and Schmidhuber, 2018, Hafner et al., 2021, 2023, Schrittwieser et al., 2020]. Program synthesis and executable-world-model work replaces the latent state with code that humans can read and tests can falsify [Ellis et al., 2021, Grand et al., 2024, Romera-Paredes et al., 2024, Guan et al., 2023, Hao et al., 2023, Wong et al., 2023, Gu et al., 2024, Levy et al., 2025, Wang et al., 2024]. Workspace optimization keeps the computational shape of model-based RL but trains a structured workspace of executable artifacts in place of a parameter vector, with feedback delivered as a counterexample owned by a named artifact rather than a gradient.

A recent line of work adapts at inference time by editing structured external state under frozen weights, with case banks [Zhou et al., 2025], distilled strategy libraries [Ouyang et al., 2025], persistent cheatsheets [Suzgun et al., 2025], and self-rewriting agent code [Zhang et al., 2025b] extending an older text-memory and skill-library line [Yao et al., 2022, Shinn et al., 2023, Packer et al., 2023, Wang et al., 2023, Park et al., 2023] that wrote into less-structured stores at trajectory or session granularity. DREAMTEAM differs on two structural axes. Failures are routed to one of a small set of typed slots whose owner is named at write time, instead of being retrieved by similarity, which turns credit assignment into an addressing decision and sidesteps the open failure-attribution problem [Zhang et al., 2025c]. Each accepted patch is replayed against earlier transitions retained as a regression set, so a revision must remain consistent with prior evidence as well as the current outcome. We give detailed related work in Appendix A.

2 Workspace Optimization

This section introduces workspace optimization, in which a frozen model accumulates task-specific competence by editing a mutable workspace of programs, traces, and logs while a harness replays earlier transitions to evaluate each edit.

Table 1: Persistent workspace components shaped by experience and read in future rounds.

Symbol	Name	What it is	Update effect
$\mathcal{P}_t$	Programs	Reusable procedures, policies, renderers, or other callable artifacts.	Edits the computations for future calls.
$\mathcal{L}_t$	Logs	Summaries, hypotheses, abandoned candidates, and unresolved questions.	Updates the agent's interpretive context.
$\mathcal{D}_t$	Trace data	Observations, actions, rewards, and counterexamples.	Appends new evidence from interaction.

2.1 The Trainable State

We model an agent as a tuple (M, H) of a language model M and a harness H providing tools, a sandbox, and a context manager. In an interactive, multi-turn environment, the harness exposes an observation dataset $\mathcal{D}_t$ and a structured **mutable workspace** W_t that the agent reads, writes, and tests around each call to M and each environment interaction. Formally, for step t:

$$W_t = (\mathcal{P}_t, \mathcal{L}_t), \qquad \mathcal{D}_t = (o_0, a_0, r_0, \ldots, o_{t-1}, a_{t-1}, r_{t-1}, o_t).$$

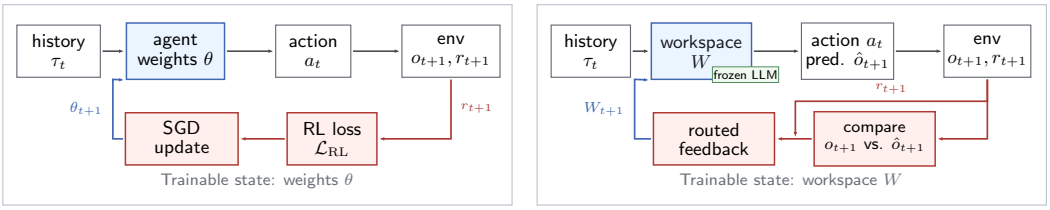
Standard agent memory is append-only: it records *what has happened* but holds no theory of what comes next, and no rules or code committed for reuse. The workspace and the observation dataset together expose three slots that the agent reads (Table 1): the append-only trace in $\mathcal{D}_t$, working hypotheses and plans in $\mathcal{L}_t$, and programs in $\mathcal{P}_t$. These components evolve across a run. At step t,

the agent reads W_t to choose an action, evaluates the outcome once the next observation o_{t+1} arrives, and writes the result back as a workspace edit:

$$a_t = \mathrm{Act}(W_t, \mathcal{D}_t), \qquad e_t = \mathrm{Evaluate}(W_t, \mathcal{D}_{t+1}), \qquad W_{t+1} = \mathrm{Update}(W_t, e_t).$$

Evaluation. The signal e_t comes from three sources. First, the *external reward* r_t, which is independent of W_t. Second, *prediction failure*: a hypothesis in $\mathcal{L}_t$ or a rule in $\mathcal{P}_t$ predicted o_{t+1}, and the next observation contradicts it. Third, *regression breakage*: the harness maintains a *regression set* $\mathcal{R}_t$, a window of past transitions drawn from $\mathcal{D}_t$, and replays the just-edited programs against every entry. A transition the previous version handled but the patched version no longer handles becomes a new counterexample for the responsible role on the next step.

2.2 The Workspace as a Computational Graph



(a) Weight-space training **(b) Workspace optimization**

Figure 2: Weight-space RL updates agent weights from interaction losses over histories, actions, environment outcomes, and rewards. Workspace optimization keeps the model frozen and updates the external state that future inference reads: artifacts, evidence, summaries, and policies. After each edit, the harness replays earlier transitions through the updated artifacts and reports the deltas as feedback.

So far we have the nodes of the workspace graph, the components of W_t, and the signal e_t that scores them, the analogues of weights and loss in standard training. What remains are the edges: how feedback at one node becomes an edit at another.

In weight-space training (Figure 2A), the chain rule distributes a scalar loss among the parameters that produced the prediction. Workspace optimization (Figure 2) does not need this machinery to name the interface that emitted a failed claim. Each prediction is emitted by one specific interface, so the prediction-failure component of e_t carries that interface's name. Interface-level attribution is therefore direct; the broader credit-assignment problem is the cascade of repairs that follows once a patch is applied. The cost of this localization, paid in advance, is committing to interfaces precise enough to be falsified.

Local attribution does not imply local repair: a patch to one interface can break earlier transitions that depend on it. Suppose a movement rule predicts that LEFT moves the avatar one cell, a wall blocks the move at step t, and the Simulator patches the rule; on replay against $\mathcal{R}_t$, the patch fails an earlier transition where LEFT did move the avatar, implicating the wall-collision rule that distinguishes the two cases. Repairs sequence themselves through replay failures, with no backward operator constructing the path in advance.

Two mechanisms substitute for differentiability. The regression set $\mathcal{R}_t$ keeps a single repair from settling on the most recent transition at the expense of older ones. The workspace's dependency structure bounds where a repair can land: each edit propagates only through calls of the edited interface, and inputs the agent cannot rewrite (the model's weights, the environment, and historical observations) terminate the chain where stop-gradient would terminate a backward pass.

Remark 1 (Initialization). Workspace optimization begins from a seed workspace W_0, analogous to weight initialization in a neural network. Appendix C describes the seed artifacts used in DREAMTEAM. The seed sets an inductive bias for how the workspace can evolve. An over-specified seed can constrain exploration and make early repairs overfit its initial ontology. An under-specified seed can slow bootstrapping and leave early behavior imprecise. A good seed gives the harness enough structure to make evidence usable while leaving room for the workspace to change.

Remark 2 (Inductive and transductive outputs). Two agents can run the same model and still differ in what they return: a value, or a callable artifact. We call the first *transductive* and the second

inductive [Vapnik, 1998, Chapelle et al., 1999]. A transductive output (a prediction, action proposal, explanation, or plan) enters $\mathcal{L}_t$ as context. An inductive output (a rule, renderer, or policy) enters $\mathcal{P}_t$ and can be invoked on inputs the agent has not seen.[2] The inductive choice is preferable when the mapping can be written as code: the function is cheap and falsifiable, at the cost of committing to a fixed interface on potentially scant evidence. The transductive choice is preferable when the input is unstructured or the answer requires the model's judgment, at the cost of less reliability and a tendency toward self-agreement.

3 ARC-AGI-3 Solver: Deriving the Architecture from First Principles

ARC-AGI-3 places a frozen language model in front of a game whose rules, mechanics, and goal are unknown. In each step t, the agent receives only a transient observation o_t and must select the next action a_t. Two features distinguish this setting from standard reinforcement-learning benchmarks. There is no offline interaction data and there is no way to rewind a step or fan out parallel rollouts to fit a policy from. They apply within a tight action-efficiency constraint: human players solve these games in a few hundred moves. Within that budget, every action is an exploration-exploitation trade-off between probing and progressing.

The frozen model brings strong arcade-and-video-game priors about what objects mean, what counts as progress, and what the goal is likely to be. When these priors align with the game's mechanics and goal they make few-hundred-action competence achievable; when they mislead, every subsequent action reinforces the wrong reading in a positive-feedback loop.

3.1 The Limits of Standard Architectures

A model-free reinforcement-learning approach has no traction here. The critic starts uninitialized with no signal to guide the policy, so progress depends on exploration alone. Even once exploration clears a level, two problems prevent the standard policy-gradient loop from closing: the trajectory is strictly online and visits each state at most once, so each rewarded transition is a single noisy example, far too weak to fit a critic; and the agent cannot return there to re-optimize, so any improvement has to transfer to states in later levels, a signal that is itself weak.

A pure model-based approach is closer to what the setting demands, since a WM can in principle be queried before each action and updated from each transition. But conventional latent-dynamics models presuppose either offline data or a long online training phase. Even state-of-the-art sample-efficient agents consume thousands of environment steps before their WMs become trustworthy, while ARC-AGI-3 expects competence after dozens. Whatever WM the agent uses must therefore be representable in a form the agent can write down, inspect, and revise.

Both routes leave a deeper problem unaddressed: without a goal hypothesis, exploration has no direction, and within a few hundred actions undirected exploration neither stumbles onto a reward signal nor covers the state space well enough for dynamics learning to converge. How can the agent acquire a goal and explore efficiently inside this budget?

3.2 The Architecture That Follows

These observations point in the same direction. The agent needs an explicit, inspectable theory of the game, constructed from interaction and tested by interaction. The theory must be detailed enough to make predictions, so that failures are diagnostic, and structured enough to be repaired locally, so that a single contradicted prediction does not require rebuilding everything.

We call this object the executable game WM and require three properties: a committed prediction before each action, so that retrodicting against the next observation either confirms or contradicts it; identifiability of the offending interface (observation parsing, hidden-state tracking, or action effect) when a prediction fails, so repairs are local; and usability inside the agent for planning once predictions are reliable, so the agent reasons in imagined rollouts as well as real actions.

This is recognizably the Dreamer template from model-based reinforcement learning [Hafner et al., 2023], with one substantive change: the WM is code rather than parameters. We call each piece of

[2] **Hybrid agents.** Some agents are both: an observer encodes "transductively" but also returns a callable renderer. (Table 2)

the workspace a *surface*: a typed slot owned by one role, holding the artifacts that role reads and edits (six in total, listed in Table 2). The training loop keeps the gradient template, with per-step error signals routed back to whichever surface emitted the failed claim. Edits patch the program in place, and the regression set $\mathcal{R}_t$ replays earlier transitions to evaluate each edit.

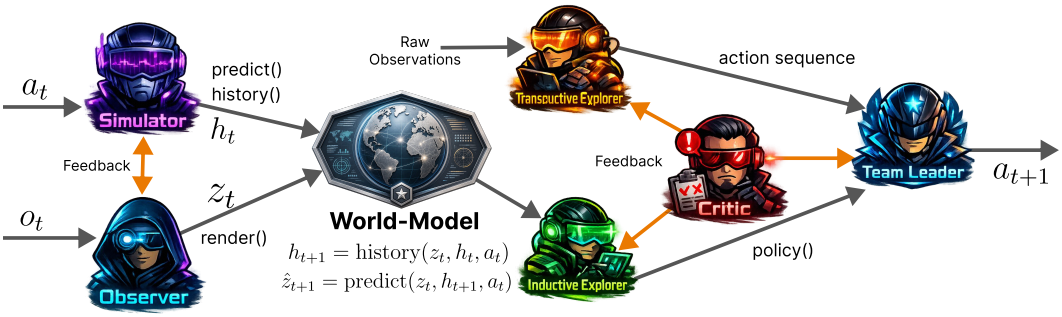


Figure 3: The DREAMTEAM role graph and executable WM interface. The Observer encodes o_t into z_t and grounds it via render. The Simulator carries history $h_{t+1} = \text{history}(z_t, h_t, a_t)$ and predicts one step ahead $\hat{z}_{t+1} = \text{predict}(z_t, h_{t+1}, a_t)$; Observer and Simulator exchange feedback when retrodiction disagrees with the next encoding ($\hat{z}_{t+1} \not\approx z_{t+1}$). The Inductive Explorer (IE) proposes a sub-goal set $\mathcal{G}_t$ and policies Π_t; the Transductive Explorer (TE) proposes action-sequence probes. The Critic emits feedback (orange edges) to the proposers and to the Team Leader (TL), who commits the next action a_t.

The remaining question, sketched in Figure 3, is who edits the WM and who uses it, and at what granularity. Too coarse: a single role carries observation, dynamics, planning, and arbitration at once, raising per-call cognitive load on every surface. Too fine: inter-role hand-offs and audits eat the per-step wall-clock budget that should pay for env actions. We settle on six roles split along two axes, maintaining the WM versus acting through it. The Observer parses observations into structured state and the Simulator tracks hidden state and patches the dynamics rule when a prediction fails; the Inductive Explorer commits reusable strategies, the Transductive Explorer proposes information-seeking probes, the Critic routes failures to their owners, and the Team Leader arbitrates which proposal is committed. The Transductive Explorer is added beyond the Dreamer template: when the WM is still too immature for rollouts to be informative, a probe-proposing role keeps early exploration directed.

4 DREAMTEAM: Executable Workspace Optimization

4.1 The Agentic Workspace

DREAMTEAM materializes the six-role decomposition of Section 3 as the team's editable workspace. Each role owns exactly one *surface*: a typed slot holding the artifacts the role reads, edits, and is held responsible for. At the center are the two WM surfaces, the *observation model* (Observer) and the *dynamics model* (Simulator); both commit a concrete claim before each action and either survive the next observation or get patched. Four further surfaces hold the action-side state used while the WM is still maturing: a *strategy library* (IE), a *probe context* (TE), *failure routing* (Critic), and the *goal and action-selection context* (TL). Table 2 lists each surface, its owner, and the artifacts written. Write W_t^r for the slice of W_t owned by role r. The surfaces interlock by design: each role's output is another role's input, so a single env step exercises the whole chain.

The WM and strategy library expose a small callable interface. The Observer parses o_t into a per-step structured state z_t, which $\text{render}(z_t)$ checks by round-tripping back to the screen. The Simulator commits to a one-step claim through $\hat{z}_{t+1} = \text{predict}(z_t, h_{t+1}, a_t)$, where h_t is an aggregated history carried across steps and updates as $h_{t+1} = \text{history}(z_t, h_t, a_t)$. The strategy library exposes a harness-side rollout that generates imagined trajectories from $\text{policy}(z_t, h_t)$. The asymmetry between predict / render and the Observer's parse is deliberate: render and predict mirror functions that exist on the game-engine side and can therefore be committed as code in $\mathcal{P}_t$ and rerun by anyone, but parsing the grid $z_t \leftarrow o_t$ has no analogue engine function, so the Observer's task is transductive: each parse produces a per-step output, and no callable form is committed for reuse.

Carrying h_t separately gives the agent traction on partially observable games: z_t alone is insufficient when the current grid hides part of the state (occlusion, off-screen entities, latent timers), and h_t is the durable summary of past observations $\mathrm{predict}$ reads alongside z_t. Full signatures, regression checks, and the role-context fields that surround these calls are deferred to Appendix C.6.

Table 2: DREAMTEAM roles, artifacts owned, and team feedback edges. The strategy library holds sub-goals $\mathcal{G}_t$ and policies Π_t used before the dynamics model is reliable; the probe context records information-seeking actions for unfit regions; failure routing turns contradicted predictions into addressable patches; the goal/action context chooses plan, probe, or repair. *Inductive output* is a callable committed to $\mathcal{P}_t$; *transductive output* is a per-step production.

Surface	**Owner**	**Inductive output**	**Transductive output**	**Feedback to**
Observation model	Obs	$\mathrm{render}()$	z_t	Sim
Dynamics model	Sim	$\mathrm{predict}()$, $\mathrm{history}()$	h_t	Obs
Strategy library	IE	$\mathcal{G}_t$, Π_t	—	—
Probe context	TE	—	action-sequence probes	—
Failure routing	Crt	—	feedback directives	IE, TE, TL
Goal/action context	TL	—	chosen action sequence or policy	—

4.2 The WM Refinement Loop

DREAMTEAM's WM refines through a commit-then-retrodict loop. The Simulator's dynamics program is fixed in W_t when action a_t is committed; after the next observation o_{t+1} arrives, the observer encodes z_{t+1} and the harness compares the prediction[3] $\hat{z}_{t+1} = \mathrm{predict}(z_t, h_{t+1}, a_t)$ with z_{t+1}:

$$e_t = \mathrm{diff}(\hat{z}_{t+1}, z_{t+1}).$$

The harness diff is one source of refinement signal. The other is *peer feedback*: roles audit each other's artifacts and route findings inline as $F_{a\to b}$, where role a writes a directed comment to role b about b's artifact. Feedback carries information the diff does not: the audit reads reasoning chains and ontology choices in addition to value mismatches. Inside the WM the Observer audits the Simulator's $\mathrm{predict}$ and the Simulator audits the Observer's z_t parse, so each maintainer's evaluation signal e_t^r bundles the diff with the incoming peer feedback, matching the $\mathrm{Update}(W_t, e_t)$ form of Section 2:

$$e_t^{\mathrm{sim}} = \big(e_t,\ F_{\mathrm{obs}\to\mathrm{sim}}\big), \qquad W_{t+1}^{\mathrm{sim}} \ =\ \mathrm{Update}\big(W_t^{\mathrm{sim}},\ e_t^{\mathrm{sim}}\big),$$

and symmetrically for the Observer with $e_t^{\mathrm{obs}} = (e_t,\ F_{\mathrm{sim}\to\mathrm{obs}})$.

In the `tu93` failure shown in Figure 4, e_t is non-empty. The harness decomposes it into named components and surfaces each to its artifact's owner: the Observer sees its per-field encoding mismatches, the Simulator sees its prediction errors. Here the diff has an observation component (the Observer had not named the obstacle sprite) and a dynamics component (the Simulator did not yet model contact as lethal); the resulting repairs add an `obstacle` entity and a lethal-contact rule. The regression checks report which previously-held cases the new program would break, informing the next round of edits.

4.3 From Probing to Planning

At each step, the three actor roles (TL, IE, TE) produce candidate actions from different signals. The IE takes the current WM state (z_t, h_t), runs $\mathrm{rollout}$ for each policy $\pi \in \Pi_t$, and scores the resulting $(\hat{z}, \hat{h})$ trajectories against the active sub-goal set $\mathcal{G}_t$; policies whose rollouts reach a sub-goal become candidate plans. The TE reads the recent step logs and the harness's retrodiction routing and proposes information-seeking action sequences for regions the WM has not yet fit. The TL sits at the sink: it inspects the IE-policy candidates and the TE-sequence candidates, judges whether the WM is reliable enough for rollout-based planning, and then commits an action or a policy as a_t.

Each actor updates from two error signals at the close of the step. First, the Critic audits every actor's proposals against the team's evidence and emits $F_{\mathrm{crt}\to\mathrm{TE}}$, $F_{\mathrm{crt}\to\mathrm{IE}}$, and $F_{\mathrm{crt}\to\mathrm{TL}}$, flagging assumption failures, ontology drift, or grinding loops. Second, the actor observes its own commitment fail: an IE policy whose rollout predicted progress and instead lost a level supplies its own counterexample,

[3]The rerun loads $\mathrm{predict}$ from the live dynamics module, so a Simulator patch made during step t is the version exercised both for this diff and for the regression replay below.

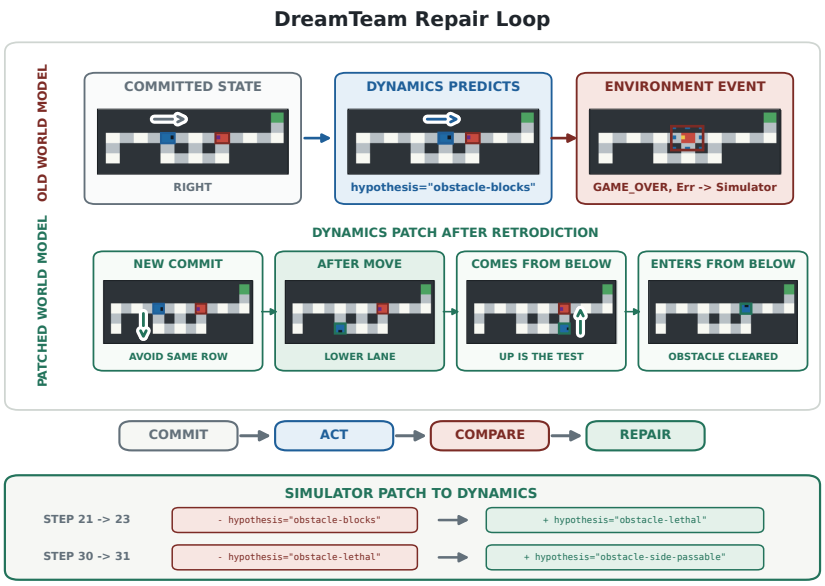


Figure 4: A commit-then-retrodict repair on a real failure (game `tu93`, level 1, step 22). The old WM commits `RIGHT` on the current board and predicts that the obstacle will block the player, but the game reaches `GAME_OVER`. The patched WM routes below the obstacle, arrives directly under it, and enters from below, avoiding the lethal same-row approach. The strip below the grid rows marks the loop, commit, act, compare, repair. The compact Simulator patch shows the hypothesis refinements that first make the obstacle lethal, then add a side-entry hypothesis.

and a TE probe whose hypothesised effect did not appear supplies one too. Both inputs enter the actor's update via a bundled evaluation signal:

$$e_t^s = \big(\delta_t^s,\ F_{\mathrm{crt}\to s}\big), \qquad W_{t+1}^s \ =\ \mathrm{Update}\big(W_t^s,\ e_t^s\big), \qquad s \in \{\mathrm{TL}, \mathrm{IE}, \mathrm{TE}\},$$

where δ_t^s is the actor's own hypothesis-vs-outcome diff, the actor-side analogue of the harness retrodiction diff that updates the WM surfaces.

The same workspace looks different at different points in a level. Early in level 1 of `tu93` (Figure 4), before the WM captured the obstacle's contact rules, actions such as touching a tile or walking into a wall served as probes: the TL had no reliable predictions to plan against. Once the surfaces stabilized and the strategy library held a tactic that fit, the TL committed to model-based plans. The first death triggered a repair that promoted obstacle contact from a blocking rule to a lethal one; for the next several steps the TL favored probes until the patch settled. The result is a control loop in which each action has an epistemic role as well as a game objective: it may confirm that the WM still fits, expose a counterexample for repair, or supply the evidence that localizes the next repair.

5 Experiments

We evaluate DreamTeam on the 25-game ARC-AGI-3 public set with the official RHAE scorer. See the ARC Prize scorecard record for one of our two runs: `https://arcprize.org/scorecards/831c83cf-b969-45fc-a6ce-27f9b3c4105c`. Figure 5(a) plots aggregate RHAE sampled every minute against a matched-budget human reference (the levels a baseline-efficient human would clear at our action count). Averaged over our two runs, DreamTeam stays ahead for the first $\sim$5 h (peak lead $\sim$+4.5 pp near hour 3); the curves cross at $\sim$5.3 h, after which the human pulls ahead through the 24-hour mark. The two runs sit in a tight band around the mean. We attribute the late-game gap to the human's stronger cross-level transfer: a mechanic understood at level ℓ is re-used nearly cost-free at level $\ell{+}1$, whereas the team rebuilds parts of its workspace at each transition.

Figure 5(b) compares per-game RHAE against Symbolica's Agentica SDK [Symbolica AI, 2026] on the same 25 games. Averaged over our two runs, DreamTeam reaches mean RHAE **38.36**% vs Symbolica's single-run 36.08%. Each level DreamTeam clears sits closer to the per-level cap,

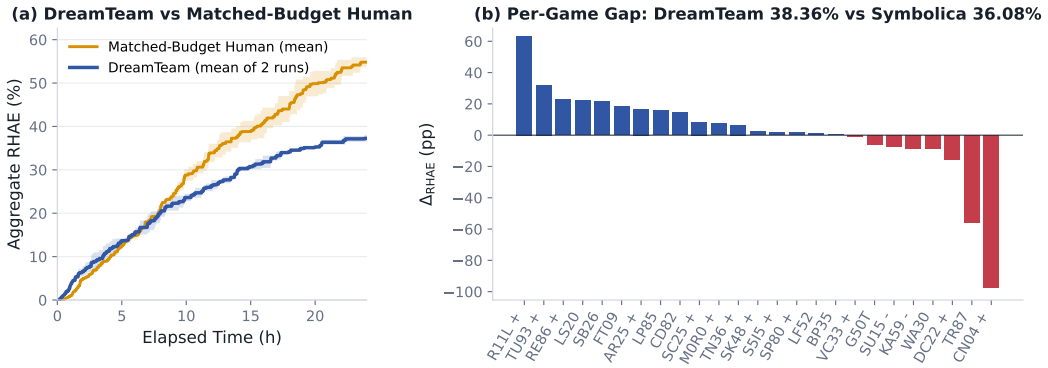


Figure 5: **(a)** 24h matched-budget RHAE timeline averaged over our two runs (per-run range shaded): DREAMTEAM (blue) vs the matched-budget human (amber). **(b)** Per-game RHAE Δ_g vs Symbolica's single published run; each bar is the per-game mean of our two runs. $+/-$ next to a label marks a baseline change in human actions (15 games have changed, see Table 10).

Table 3: Per-role activity per env_step, averaged over all 22,180 env_steps across 25 games and our two independent runs. *LLM* = LLM calls; *In/Out K* = input/output tokens (thousands); *Fb s/r* = peer @-mentions sent/received per step; *Gen s* = LLM generation seconds; *Code* = REPL blocks executed. *TOTAL* sums per-role columns; *Gen s* on the TOTAL row is the wall-clock seconds per step (not the sum of per-role generations, since roles run in parallel).

Role	Model	LLM	In K	Out K	Fb s/r	Gen s	Code
Observer	Opus 4.6 (none)	1.97	133.1	3.6	1.1/2.9	68.5	5.20
Simulator	Opus 4.6 (none)	1.93	98.8	4.0	1.0/5.9	82.9	5.82
Inductive Explorer	GPT-5.5 (high)	1.92	140.2	5.1	3.2/8.7	81.7	3.17
Transductive Explorer	Opus 4.6 (low)	1.75	75.5	4.5	0.9/5.7	147.8	2.83
Critic	GPT-5.5 (high)	1.95	154.1	4.3	22.8/2.0	76.8	2.83
Team Leader	Opus 4.6 (low)	2.01	91.3	4.5	5.3/9.3	159.2	2.49
TOTAL	n/a	**11.5**	**692.9**	**26.0**	**34.4/34.4**	**205.2**	**22.3**

which the triangular weighting amplifies. On the ten games whose version did not change in the April 14 update the gap is $+3.42$ pp in our favor.

DREAMTEAM's RHAE gain over Symbolica comes with substantially fewer environment actions. Across the 25-game set and averaged over our two runs, DREAMTEAM consumes a mean of **444** environment actions per game against Symbolica's single published run of **16,081**, a **31**% reduction in mean per-game actions. The largest gains come from games where the baseline spends many actions on a single level (whether the level is eventually cleared or not): showcasing the advantage in planning over a learned world model. Restricted to successful levels and pooled across our two runs, DREAMTEAM averages **54.1** actions per solved level vs Symbolica's **100.1**, a **46**% reduction; on the 85 level indices that all of run 1, run 2, and Symbolica cleared, the per-level counts are **49.0** (DT mean of two runs) vs **57.3** (-14.5%). Table 3 reports per-role activity over the 22,180 steps pooled across our two runs; see full details in Appendix E.

6 Limitations and Conclusion

Our reported scores are bounded by time and cost budgets rather than by system intelligence: the wall-clock cap sets the ceiling for many games we did not finish, and a larger budget would plausibly lift the headline numbers further. A second infrastructure limitation is our reliance on hosted provider APIs, whose latency and availability can drift over the course of a run. Overloaded endpoints surface as timeouts and degraded per-step throughput; although these effects are external to the agent design they still materially affect any single-run number.

These are operational caveats. The conceptual claim of the paper concerns what becomes trainable once weights are off the table, and DREAMTEAM makes that state explicit and executable: every

prediction is a written commitment, every contradicted commitment becomes a typed repair, and every repair is replayed against earlier evidence before it is accepted. Treating the workspace as the object of training opens a research agenda parallel to the one that has organized weight-space learning for decades, with its own questions about which artifacts to expose, which counterexamples to retain, and which repair operators generalize across environments. The present results are a first step along that agenda, and we expect the methodology to extend to other interactive settings where the model is fixed and learning has to happen through the workspace around it.

References

Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Anthropic. Claude code. `https://www.anthropic.com/claude-code`, 2025. Accessed 2026-05-06.

ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026a.

ARC Prize Foundation. ARC-AGI-3 API documentation: Changelog. `https://docs.arcprize.org/changelog`, 2026b. April 14, 2026 entry: 15 game-version updates, median-human baseline, per-level cap raised to 1.15×, 24-hour scorecard cap.

Bernard J. Baars. Global workspace theory of consciousness: Toward a cognitive neuroscience of human experience. *Progress in Brain Research*, 150:45–53, 2005.

Jeremy Berman. How I got the highest score on ARC-AGI again swapping Python for English. Blog post, `https://jeremyberman.substack.com/p/how-i-got-the-highest-score-on-arc-agi-again`, September 2025. Reports 29.4% on ARC-AGI-2 via evolutionary test-time compute over natural-language instruction candidates. Accessed 2026-05-06.

Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.

Olivier Chapelle, Vladimir Vapnik, and Jason Weston. Transductive inference for estimating values of functions. In *Advances in Neural Information Processing Systems*, volume 12, 1999.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023. ICLR 2024.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI*, 2021.

Gabriel Grand, Lionel Wong, Matthew Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. LILO: Learning interpretable libraries by compressing and documenting code. *arXiv preprint arXiv:2310.19791*, 2024. ICLR 2024.

Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, Huan Sun, and Yu Su. Is your LLM secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2024. TMLR 2025.

Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. *arXiv preprint arXiv:2305.14909*, 2023. NeurIPS 2023.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. NeurIPS 2018.

Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2021. ICLR 2021.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023. Published in Nature, 2025.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992*, 2023. EMNLP 2023.

Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. In *ICLR*, 2024.

Matthew Ho, Chen Si, Zhaoxiang Feng, Fangxu Yu, Yichi Yang, Zhijian Liu, Zhiting Hu, and Lianhui Qin. ArcMemo: Abstract reasoning composition with lifelong LLM memory. *arXiv preprint arXiv:2509.04439*, 2025. Frozen-weight test-time adaptation: distills concept-level abstractions from solution traces and retrieves them for new ARC queries.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023. ICLR 2024.

Greg Kamradt. Analyzing GPT-5.5 and Opus 4.7 with ARC-AGI-3. ARC Prize Foundation blog, `https://arcprize.org/blog/arc-agi-3-gpt-5-5-opus-4-7-analysis`, May 2026. Published 2026-05-01. 160-replay failure-mode analysis on the released benchmark.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Guillaume Levy, Cedric Colas, Pierre-Yves Oudeyer, Thomas Carta, and Clement Romac. WorldLLM: Improving LLMs' world modeling using curiosity-driven theory-making. *arXiv preprint arXiv:2506.06725*, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023. NeurIPS 2023.

Stefan Nielsen, Edoardo Cetin, Peter Schwendeman, Qi Sun, Jinglue Xu, and Yujin Tang. Learning to orchestrate agents in natural language with the Conductor. *arXiv preprint arXiv:2512.04388*, 2025. ICLR 2026.

OpenAI. OpenAI codex. `https://openai.com/codex`, 2025. Accessed 2026-05-06.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025. Accepted to ICLR 2026. Distills generalizable reasoning strategies from self-judged successes and failures; introduces memory-aware test-time scaling.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. *arXiv preprint arXiv:2411.13543*, 2024. ICLR 2025.

Dongmin Park, Minkyu Kim, Beongjun Choi, Junhyuck Kim, Keon Lee, Jonghyun Lee, Inkyu Park, Byeong-Uk Lee, Jaeyoung Hwang, Jaewoo Ahn, Ameya S. Mahabaleshwarkar, Bilal Kartal, Pritam Biswas, Yoshi Suhara, Kangwook Lee, and Jaewoong Cho. Orak: A foundational benchmark for training and evaluating LLM agents on diverse video games. *arXiv preprint arXiv:2506.03610*, 2025. ICLR 2026.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. UIST 2023.

Julien Pourcel, Cédric Colas, and Pierre-Yves Oudeyer. Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. In *ICML*, 2025. arXiv:2507.14172. Introduces SOAR; reports 52% on the ARC-AGI public test by alternating evolutionary program search with hindsight LLM fine-tuning.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

Evgenii Rudakov, Jonathan Shock, and Benjamin Ultan Cowley. Graph-based exploration for ARC-AGI-3 interactive reasoning tasks. *arXiv preprint arXiv:2512.24156*, 2025. AAAI 2026 Workshop.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588: 604–609, 2020.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. NeurIPS 2023.

Dries Smit. 1st place in ARC-AGI-3 agent preview competition. `https://medium.com/@dries.epos/1st-place-in-the-arc-agi-3-agent-preview-competition-49263f6287db`, 2026.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *ICML*, 2020.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*, 2025. Frozen LLM with a persistent evolving cheatsheet of strategies, code, and insights; doubles Claude 3.5 on AIME and lifts GPT-4o on Game-of-24 from 10% to 99%.

Symbolica AI. Arcgentica: Game-agnostic multi-agent harness for ARC-AGI-3. `https://github.com/symbolica-ai/ARC-AGI-3-Agents`, 2026.

Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. *arXiv preprint arXiv:2408.14837*, 2024. ICLR 2025.

Vladimir N. Vapnik. *Statistical Learning Theory*. John Wiley & Sons, 1998.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. NeurIPS 2023.

Ruocheng Wang, Eric Zelikman, Gabriel Poesia, Yewen Pu, Nick Haber, and Noah D. Goodman. Hypothesis search: Inductive reasoning with language models. *arXiv preprint arXiv:2309.05660*, 2024. ICLR 2024.

Lionel Wong, Gabriel Grand, Alexander K. Lew, Noah D. Goodman, Vikash K. Mansinghka, Jacob Andreas, and Joshua B. Tenenbaum. From word models to world models: Translating from natural language to the probabilistic language of thought. *arXiv preprint arXiv:2306.12672*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. ICLR 2023.

Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models. *arXiv preprint arXiv:2512.24601*, 2025a.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025b. Self-improving system that iteratively rewrites its own coding-agent code, validated empirically; SWE-bench 20% to 50%.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *ICML*, 2025c. Spotlight. arXiv:2505.00212. Introduces the Who&When dataset; current attribution methods reach only 53.5% agent-level accuracy.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*, 2025. Planner/executor with a case bank of past trajectories retrieved by a learned policy; formalized as a memory-augmented MDP. Tops GAIA without weight updates.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. *arXiv preprint arXiv:2402.16823*, 2024. ICML 2024.

A Extended Related Work

Model-based reinforcement learning and neural world models. Model-based RL learns predictive state and dynamics and then uses the resulting model for planning or policy improvement. World Models, Dreamer, and MuZero do this with neural latent states, learned transition models, and imagined rollouts [Ha and Schmidhuber, 2018, Hafner et al., 2021, 2023, Schrittwieser et al., 2020]. The thing being trained, in all of these, is a parameter vector. DREAMTEAM keeps the computational shape (encode, maintain state, predict, reconstruct, roll out), but the substrate is code rather than weights, and feedback is a logged counterexample with a named owner rather than a gradient.

Generative interactive environments and playable world models. A recent line of generative-model work builds world models that are explicitly *playable*: a user or policy issues actions and the model produces the resulting observations frame by frame in real time. Genie trains an action-conditioned video model that turns static internet videos into controllable interactive environments [Bruce et al., 2024]. GameNGen demonstrates that a diffusion model can serve as a real-time game engine for DOOM, with neural rollouts standing in for the simulator [Valevski et al., 2024]. DIAMOND uses diffusion world models on Atari and shows that visual fidelity inside the model materially affects what a policy trained inside it can learn [Alonso et al., 2024]. DREAMTEAM's world model is playable in the same functional sense. The Simulator commits to one-step predictions through `predict`, the strategy library composes those predictions into multi-step `rollouts`, and the

Inductive Explorer evaluates candidate sub-goals by playing them out inside the model before any real action is taken. The substrate differs from the generative-interactive-environments line: pixels rendered from a weight vector trained offline on large video corpora in their case, a small set of typed Python artifacts read, edited, and regression-tested online from at most dozens of interactions in ours. The shared design point is that the world model is the place where rollouts happen, and learning targets the fidelity of those rollouts to the real environment.

LLM agents with memory, tools, and code artifacts. LLM agents already adapt at inference time, through action traces, retrieved memories, reflections, tool calls, cognitive-architecture state, and persistent skill code [Yao et al., 2022, Shinn et al., 2023, Packer et al., 2023, Sumers et al., 2023, Wang et al., 2023]. The writable state in most of this work is some combination of transcript, memory store, tool result, and skill library. The standard handling of a failure is to append another message, summary, reflection, or document to that pool. DREAMTEAM differs in where the failure goes: a misparse is sent to the observation model and not to anything else, a wrong action effect to the transition model, a tactic that overfits its own progress check to the strategy library, and a failure with no clean owner is converted into a probe. The point is the routing, not the size of the context. Meta-Harness [Lee et al., 2026] optimizes the harness itself by searching over harness code across tasks via an agentic proposer that reads prior candidates' source, scores, and traces; workspace optimization runs the inner loop atop a fixed harness, adapting W_t within a single episode.

Text-memory agents. A natural competitor class to typed-artifact workspaces is agents whose memory is mediated by natural-language text rather than typed, executable artifacts. Voyager grows a library of code skills retrieved by their natural-language descriptions, with routing done by description match rather than by artifact type [Wang et al., 2023]. Reflexion appends a verbal self-critique after each episode and reads it back as a textual memory that conditions the next attempt [Shinn et al., 2023]; the unit of update is a completed trajectory, so feedback only becomes available between episodes. Generative Agents organize experience as a memory stream of natural-language observations scored by importance and recency at retrieval time [Park et al., 2023]. MemGPT borrows operating-system metaphors to manage hierarchical text memory across a fast working context and a slower archival store [Packer et al., 2023]. What unifies these systems is that memory is natural-language text retrieved at use time, where a passage is the unit of both write and read. DREAMTEAM instead writes into typed, executable artifacts: a contradicted commitment is routed to the observation, dynamics, or strategy slot that owns it, the patch is replayed against earlier evidence in a regression set, and credit assignment names a typed address rather than a passage to retrieve. The update cadence is also finer. Each environment action carries a one-step prediction that the next observation either confirms or contradicts, so a workspace edit is proposed within the trajectory itself, at the granularity of an individual transition. Adaptation therefore proceeds during the episode, and the regression set of earlier transitions surfaces consistency failures between candidate patches and prior evidence.

Programmatic context interaction and recursive inference. Recursive Language Models [Zhang et al., 2025a] place a long prompt inside a Python REPL and let a root LLM examine it through code, optionally spawning recursive sub-calls on snippets and returning a value via `FINAL` or a REPL variable. The prompt becomes external state the model interrogates rather than tokens it must read. We share the move of pushing structure outside the model and letting code mediate between the model and a large external object. The settings differ in three ways. First, RLM operates within a single query and its REPL is ephemeral, while workspace optimization persists W_t across steps and treats it as the trainable object. Second, RLM produces a transductive output (a value for the current query) and has no mechanism by which one call's failure localizes a repair that later calls will read; DREAMTEAM routes each contradicted commitment to a named owner and evaluates the patch against a regression set. Third, RLM decomposes one prompt with copies of the same model, while DREAMTEAM decomposes the agent itself into roles owning distinct surfaces (observation, dynamics, strategy, probing, critique, arbitration). RLM is a clean instance of a transductive-only design in our taxonomy of Section 2.

Program synthesis and executable world models. Program synthesis and library-learning systems treat programs as learned structure revised by examples, tests, or search [Ellis et al., 2021, Grand et al., 2024, Romera-Paredes et al., 2024]. Related LLM systems write planning domains, propose and refine world models from interaction, search over candidate hypotheses, or compile planning structure [Guan et al., 2023, Hao et al., 2023, Wong et al., 2023, Gu et al., 2024, Levy et al., 2025,

Wang et al., 2024]. On ARC-AGI-1/2, frozen-LLM agents evolve a candidate solution against demonstration grids, with Python programs on ARC-AGI-1 and natural-language instructions on ARC-AGI-2 [Berman, 2025]. SOAR scales the same family by alternating evolutionary program search with hindsight LLM fine-tuning, reaching 52% on the ARC-AGI public test [Pourcel et al., 2025]; the inner search loop is similar to Berman's, but the proposer's weights are updated between trials, so adaptation is split between the candidate workspace and the parameter vector. Both designs run offline against demonstration sets, where the demonstration set serves as the regression evaluator and the candidate is the workspace. DREAMTEAM is closest to this line through its Simulator artifact, which evolves online against the regression set in the same way that offline candidate programs evolve against demonstration sets. The distinction is online control: each environment action produces a counterexample or confirmation, and a repair targets the current prediction while being evaluated against earlier traces. The executable world model is both the planning object and the object trained within the episode.

Online world modeling with curiosity-driven hypothesis revision. WorldLLM constructs a frozen-model world model online by maintaining a natural-language hypothesis about environment dynamics, refining it through Bayesian inference with a second LLM serving as the proposal distribution, and gathering evidence with an RL policy whose reward is the negative log-likelihood of observed transitions under the current predictor [Levy et al., 2025]. The setting is close to DREAMTEAM: both keep the predicting LLM frozen, both treat the world model as the primary trainable surface, and both use a secondary LLM to propose updates from accumulated evidence. The two systems differ along three axes. First, the substrate. WorldLLM stores the world model as a single passage of natural-language hypothesis read by the predictor at inference time. DREAMTEAM stores it as executable Python with explicit `predict` and `rollout` entry points, so multi-step rollouts are evaluated by the runtime, and the workspace is partitioned into typed slots for observation, dynamics, strategy, probing, critique, and arbitration. Second, the verification of an update. WorldLLM accepts a revised hypothesis when it improves likelihood under the current evidence buffer. DREAMTEAM replays a loaded patch against prior transitions retained as a regression set and reports any consistency failures as counterexamples, so the check is explicit and falsifiable even though replay is feedback rather than an automatic rollback gate. Third, the cadence. WorldLLM alternates evidence collection with hypothesis revision in trials. DREAMTEAM proposes a patch after every action and treats the next observation as a one-step counterexample or confirmation. In our taxonomy, WorldLLM is a single-slot instance of workspace optimization in which the proposer is Bayesian, the verifier is likelihood, and the cadence is trial-level.

Iterative self-correction with execution feedback. A line of LLM-agent work treats outputs as drafts that the model revises against feedback from a critic or an executor. Self-Refine prompts the same model to critique and rewrite its own answer over multiple iterations [Madaan et al., 2023], and Self-Debug pairs code generation with execution traces so that the model can localize and repair the offending lines [Chen et al., 2023]. These systems share with DREAMTEAM the move of routing a counterexample back into the artifact that produced it. DREAMTEAM differs in two structural respects. First, the artifact under repair is one of a small set of typed surfaces (observation, dynamics, strategy, probing) with bounded scope, so a contradicted commitment names a specific owner rather than revising a single prompt or program in place. Second, each loaded patch is replayed against earlier transitions retained as regression tests, so any disagreement with prior evidence is surfaced as explicit feedback alongside the current failure.

Test-time training and online adaptation under fixed parameters. Test-time training adapts a model on the test instance itself by running self-supervised updates before producing a prediction [Sun et al., 2020]. The strategy has been unusually effective on abstract-reasoning benchmarks. Fine-tuning on synthetic transformations of each ARC task before solving it raises accuracy substantially [Akyürek et al., 2024], and nearest-neighbor variants improve language-model reasoning more broadly [Hardt and Sun, 2024]. More recent variants pursue the same goal under frozen weights: ArcMemo distills concept-level abstractions from prior solution traces into a lifelong memory retrieved at test time, reporting consistent gains on ARC-AGI without parameter updates [Ho et al., 2025]. These methods address the same problem DREAMTEAM targets: how a model adapts to an unfamiliar instance in the absence of offline data or task-specific rewards. The mechanism differs along two axes. First, gradient-based test-time training writes the update into the parameter vector, which requires weight access and discards the adaptation once the task ends, while memory-retrieval variants grow a flat

pool of past abstractions accessed by similarity to the current input. DREAMTEAM instead writes the update into a typed workspace whose slots persist as inspectable text and code and survive deployment behind a fixed inference API. Second, the test-time-training update is driven by a self-supervised auxiliary objective derived from the test input, and the memory-retrieval update is driven by a similarity score, while the DREAMTEAM update is driven by a contradicted commitment routed to the typed artifact responsible and scored against a regression set, so credit assignment names a structural owner instead of producing a global parameter direction or a retrieval ranking.

Multi-agent LLM systems and computational graphs. Multi-agent LLM systems assign roles, exchange messages, decompose tasks, or optimize communication graphs [Hong et al., 2023, Zhuge et al., 2024, Guo et al., 2024]. Conductor [Nielsen et al., 2025] learns an orchestrator's weights via reinforcement learning over worker topologies and per-worker prompts, while DREAMTEAM keeps every model frozen and locates learning in the routed patch and the regression set. Recent work formalizes the inverse direction, automated failure attribution: given a multi-agent failure, identify which agent and which step caused it [Zhang et al., 2025c]. Current methods on the Who&When benchmark reach only 53.5% agent-level accuracy, which suggests that attribution is hard when agents communicate over unstructured natural-language messages and any agent could in principle be responsible for any commitment. DREAMTEAM sidesteps the inverse problem by construction: every prediction is emitted by exactly one typed slot, so the responsible owner is named at write time and need not be reconstructed afterwards. DREAMTEAM also uses roles, but as nodes in a workspace training graph. Each node owns part of the trainable state or update process: observation, dynamics, strategy, probing, critique, or arbitration. A prediction failure is not debate material. It is routed to an owner with bounded patch scope, and the regression set is replayed after the edit, with deltas surfaced as feedback for subsequent revisions.

ARC and interactive reasoning benchmarks. ARC-style benchmarks test abstraction and rule discovery under weak priors [Chollet, 2019]. ARC-AGI-3 extends this pressure to interaction, where the task is hidden in the game and every exploratory action consumes budget [ARC Prize Foundation, 2026a]. Game-based agent benchmarks also stress planning, observation, and tool use [Paglieri et al., 2024, Park et al., 2025]. DREAMTEAM targets the narrower bottleneck of within-episode construction of a testable world model.

The released ARC-AGI-3 benchmark reports frontier-LLM baselines below one percent of human RHAE: Gemini 3.1 Pro at 0.37%, GPT-5.4 at 0.26%, Opus 4.6 at 0.25%, and Grok 4.20 at 0% [ARC Prize Foundation, 2026a]. A subsequent 160-replay analysis of GPT-5.5 and Opus 4.7 traces these scores to a missing structured world model: agents register local action effects without composing them into a stable theory of game dynamics, with Opus over-committing to a wrong theory and GPT-5.5 failing to compress evidence into a theory at all [Kamradt, 2026]. The strongest published agent on the released benchmark is Symbolica's Agentica SDK, an Opus 4.6 multi-role harness that reaches 36.08% RHAE on 113 of 182 levels at approximately \$1,005 [Symbolica AI, 2026], while DREAMTEAM reaches 38.06% on the post-update protocol described in Section 5. The earlier ARC-AGI-3 Agent Preview Competition produced complementary design points, including a non-LLM reinforcement-learning baseline [Smit, 2026] and a graph-based exploration agent [Rudakov et al., 2025], both reported on the Preview environment and on game versions that predate the April 14 update [ARC Prize Foundation, 2026b], placing them outside the released-benchmark protocol used in our results.

B Algorithm and Pseudocode

This appendix expands the operational detail of the per-step loop summarized in Algorithm 1. Section B.1 traces the full one-step sequence; Section B.2 describes the reset and level reflection flows that wrap the step loop; Section B.3 describes role scheduling and credit assignment by ownership; Section B.4 specifies validation, regression, and the behavior of a failed repair.

Notation and concrete defaults. Throughout, we use $W_t = (\mathcal{P}_t, \mathcal{L}_t)$ for the workspace at step t, with $\mathcal{P}_t$ holding its three executable artifacts (`observable.py`, `dynamics.py`, `strategy.py`) and $\mathcal{L}_t$ holding the per-role Markdown logs (step log and level log). Alongside the workspace, the harness maintains the observation dataset $\mathcal{D}_t$ as JSONL traces (z_{encoded}, h_{encoded}, `level_constants`), and $\mathcal{R}_t$ is the regression set drawn from $\mathcal{D}_t$. Per-step quantities z_t, h_t, a_t, and c_ℓ refer to the parsed scene,

the dynamics hidden state available at observation o_t, the committed action, and the level constants. Thus the transition ending at o_t is evaluated as $\mathrm{predict}(z_{t-1}, h_t, a_{t-1}, \ldots)$, with h_t produced by $\mathrm{history}(h_{t-1}, z_{t-1}, a_{t-1}, \ldots)$. We use K for the catch-up bound on h-replay, d for the per-role round depth, and B for the per-step token budget. Unless overridden by the run configuration, runtime defaults are: round depth $d = 4$ per role, hidden-state catch-up bound $K = 20$ steps, RESET cooldown 5 actions, error-ledger capacity 20 entries per source per role, step wall-clock timeout 360 seconds, per-step token budget $B = 250{,}000$ tokens across all roles, step-log retention of the last 24 appended sections, and a per-game cost cap of \$2,000 (USD) above which the run terminates. Per-role launch delay defaults to one step for the Critic and Inductive Explorer and to zero for the other four roles.

Algorithm 1 One DREAMTEAM timestep

Require: New o_t and a Workspace W_t with logs $\mathcal{L}_t$ and the programs $\mathcal{P}_t$ in Table 2
Ensure: Action sequence $\{a_i\}$ or $\mathrm{policy}_j()$

1: **Observer**: emit transient z_t; check $\mathrm{render}(z_t) \approx o_t$
2: **Simulator**: reshape predict and history so that $\mathrm{predict}(z_{t-1}, h_t, a_{t-1}) \approx z_t$
3: **TE**: analyze the grid, write hypotheses, and propose an information-seeking action sequence
4: **for** policy $\pi_i \in \Pi$ **do**
5: **IE**: $(\hat{z}, \hat{h})^{\pi_i}_{t+1:t+k} \leftarrow \mathrm{rollout}(z_t, h_t, \pi_i)$; score against $\{\mathrm{sg}_j\}$
6: **end for**
7: **Critic**: audits routed evidence and sends feedback $F_{\mathrm{Crt}\to s}$ to actors; each role updates its logs and programs
8: **TL**: select action sequence (from **TE**) or policy (from **IE**)

B.1 One-step operational sequence

A timestep proceeds in ten phases. Phases (1) through (4) are harness-side preparation, phases (5) through (8) are role activity, and phases (9) through (10) commit the action and route the resulting evidence. The harness is a deterministic state machine; the only stochastic component within a step is the language-model call inside each role. Algorithm 2 states the same sequence as pseudocode; the prose below explains what each phase does and why.

Phase 1: Step context construction. The harness assembles a step context that captures the current grid o_t, action history, level index, level-start step, and per-step metadata (available actions, action budget, diff pixel count from the previous transition). On a soft RESET, prior z_{encoded} entries for the current level are restored so that downstream readers bootstrap from level start rather than from an empty state.

Phase 2: Hidden state recovery. The dynamics hidden state h_t is loaded by an authoritative reader. A direct hit on h_{encoded} at step t returns $(h_t, 0)$. A miss triggers replay from the nearest anchor in the window $[t-K, t-1]$, with the catch-up bound set to $K = 20$ steps by default. For each intermediate step s, the harness computes

$$h_s = \mathrm{history}(h_{s-1},\, z_{s-1},\, a_{s-1},\, c_\ell,\, m_s),$$

where c_ℓ are the level constants and m_s is the per-step metadata. The transition with $a_{s-1} = \texttt{RESET}$ is handled by setting $h_s \leftarrow \emptyset$, matching the convention used by the predict-side evaluator. Replay sources history from the live `dynamics` module, so the most recent Simulator patch is the one actually exercised.

Phase 3: Live world-model evaluation. For $t > 0$ and when both Observer and Simulator are present, the harness runs a one-step retrodiction on the previous transition. It loads z_{t-1}, h_t, and the action a_{t-1} that the Team Leader committed at the prior step, then computes

$$\hat{z}_t = \mathrm{predict}(z_{t-1},\, h_t,\, a_{t-1},\, c_\ell,\, m_t)$$

and the field-level diff $e_{t-1} = \mathrm{diff}(\hat{z}_t,\, z_t)$. A second pass evaluates each named hypothesis in `HYPOTHESES` under the same input, recording per-hypothesis prediction accuracy that is later used by the rollout-readiness check. Any error raised during predict, history, or rollout is tagged with an owner hint: predict and history failures are routed to the Simulator, POLICIES and SUB_GOALS failures to the Inductive Explorer, and render failures to the Observer.

Algorithm 2 One DREAMTEAM timestep, expanded.

Require: Workspace $W_t = (\mathcal{P}_t, \mathcal{L}_t)$ and observation dataset $\mathcal{D}_t$; environment with action set $\mathcal{A}_t$; observation o_t; committed previous action a_{t-1}; roles $\mathcal{R} = \{\text{Obs}, \text{Sim}, \text{IE}, \text{TE}, \text{Crit}, \text{TL}\}$; hyperparameters (d, K, B).
Ensure: Workspace W_{t+1}, action a_t, transition event τ_t.

Harness preparation.
$\text{ctx}_t \leftarrow$ BUILDSTEPCONTEXT$(o_t, a_{t-1}, \text{level}_t, \text{metadata}_t)$
$(h_t, k_t) \leftarrow$ LOAD$(h_t,\ K)$ # direct hit $(h_t, 0)$ or replay $k_t \in [1, K]$ steps
if $t > 0$ **and** Observer and Simulator are present **then**
  $\hat{z}_t \leftarrow \text{predict}(z_{t-1}, h_t, a_{t-1}, c_\ell, m_t)$
  $e_{t-1} \leftarrow \text{diff}(\hat{z}_t, z_t)$; attach owner tag per error type
  Update per-hypothesis retrodiction accuracy in `HYPOTHESES`
end if
Role activity.
Determine eager and deferred roles from start-after gates $\mathcal{G}$
for $r \in \mathcal{R}$ eager **do**
  Build role message $I_{r,t}$ from canonical block list; launch with stagger
end for
for $r \in \mathcal{R}$ deferred **do**
  Wait on $\mathcal{G}_r$; rebuild $I_{r,t}$ post-gate; launch
end for
for each role r **do**
  for $\rho \leftarrow 1$ **to** d **do**
    Run REPL round; apply tool calls (read, write, append, edit)
    for each write to $\mathcal{P}_t$ **do**
      PARSEVALIDATE; RELOADMODULE; LIVEREVALIDATE
    end for
    if output emitted **or** token cap B reached **then**
      break
    end if
  end for
end for
Team Leader: wait gate $\rightarrow$ read teammate logs $\rightarrow$ action validation $\rightarrow$ emit a_t
if a_t is invalid **or** no answer **then**
  $a_t \leftarrow$ CHOOSEDEFAULTACTION(preferred = `None`)
end if
Commit and route.
$o_{t+1}, r_{t+1}, \text{done}_{t+1}, \text{info}_{t+1} \leftarrow \text{env.step}(a_t)$
Suppress diff if info_{t+1} reports a level transition
$\tau_t \leftarrow$ BUILDTRANSITIONEVENT$(a_t, o_t, o_{t+1}, \text{info}_{t+1})$; log to $\mathcal{D}_t$
UPDATEERRORLEDGER: replay retrodiction across recent transitions; mark resolutions
Append step-log sections; trim to retention cap; save chats
return W_{t+1}, a_t, τ_t

Phase 4: Concurrent role launch with dependency ordering. All six roles are launched concurrently, but two start-after edges defer specific roles until upstream writes are visible:

- Simulator defers until Observer has appended its current-step z_{encoded}.
- Inductive Explorer defers until Observer has appended its current-step z_{encoded}.

The dependency is expressed as a tool-gate of the form `append_file:data/z_encoded.jsonl:observer`, not as agent-completion. Tool-gate semantics are stronger than role-termination because a role can finish before its file write becomes visible to a downstream reader. Each role is also subject to a per-role launch delay (in steps) and a per-role stagger (in seconds) that smooths gateway load. Critic and Inductive Explorer use the default launch delay of one step, so Observer and Simulator have produced z_{encoded} and h_{encoded} before either evaluates the new scene. On step zero, a wait gate auto-skips required roles that are still delayed, so the Team Leader does not block on an absent Critic at game start.

Phase 5: Per-role context assembly. Each role's first-round message is assembled from a canonical, role-specific list of context blocks ordered as primacy, middle, recency. The primacy zone holds the step directive, level-transition status, round budget, and grid history. The middle zone holds analytical inputs (world-model status, retrodiction summary, render quality, metadata, teammate logs, asynchronous chat). The recency zone holds the role's own working materials (its executable artifact, its helper file, its level log, its step log). The call to action (workflow, checklist, output format) sits at the end so that the format constraint is closest to the model's emission point. The exact block order per role is fixed in the role-context registry.

Phase 6: Bounded-depth REPL rounds. Each role runs for at most $d = 4$ rounds. A round consists of a model call, optional REPL execution, and tool calls (read, write, append, edit). Within a round the role can iterate freely over its REPL; between rounds, middleware emits structured update blocks reflecting any change to the live world-model surface. Rounds end when the role emits its required output (typically a step-log append plus, where applicable, a code edit) or when the depth bound is reached. A token budget ceiling of $B = 250{,}000$ tokens applies across all roles in the step, and the default wall-clock timeout of 360 seconds applies to the entire step (the analyzed run in Appendix E disables this cap); whichever fires first triggers a forced collection in which the harness uses whatever each role has produced.

Phase 7: Workspace patches with live validation. Patches to $\mathcal{P}_t$ (`observable.py`, `dynamics.py`, `strategy.py`) are applied via `write_file` or `edit_file`. Three layers act on each patch:

1. Python source validation rejects writes that fail to parse and reports the syntax error back to the caller.
2. World-model loader middleware re-imports the patched module into `sys.modules` so subsequent harness operations see the new functions.
3. World-model validation middleware reruns predict and render on the latest transition under the new code and emits a delta block that names which test cases passed or failed. The delta is feedback rather than a hard gate, so the role can keep iterating within the round budget.

Patches to text artifacts (step log, level log, helper) and JSONL data files (z_{encoded}, h_{encoded}, level constants) are appends rather than overwrites. The harness deduplicates JSONL writes that repeat the most recent record.

Phase 8: Team-leader arbitration. The Team Leader runs a wait gate that holds until the required upstream roles have either produced output or been excused by the delay logic. It then reads each role's current step-log section through a tail-follow channel, applies action validation (the action must be in the available-actions list and respect the RESET cooldown of 5 actions), applies policy-readiness validation (refuse to commit to a strategy rollout when the underlying world model has insufficient retrodiction support), and emits the action commitment in a structured output block. If the Team Leader fails to produce a valid answer within its safety round, the controller routes through the default-action policy specified in Section B.2, which is structured specifically to avoid implicit RESETs.

Phase 9: Environment step. The harness invokes $\mathrm{env.step}(a_t)$ and receives o_{t+1}, reward, done, and $\mathrm{diff_pixels}$. When the action triggers a level transition, the cross-level diff is suppressed (set to `None`) so that new-level pixels do not leak into the previous level's evidence. The harness logs a transition event with frame indices, frame decimation, and the final frame of the completed level when the SDK returns intermediate frames.

Phase 10: Counterexample routing and ledger update. The transition event is the canonical evidence for the next step. The error ledger middleware reruns retrodiction across the recent action history under the latest code and records, for each violation, a tuple $(\mathrm{source}, \mathrm{step}, \mathrm{fields_wrong}, \mathrm{action})$ where `source` is either `render` (Observer-owned) or `predict` (Simulator-owned). New entries are announced to the owning role at the next step through a single-shot WM Update block. Ledger entries that pass under the latest code are marked `resolved_at` and the role sees a one-line resolution notice exactly once. The persistent ledger surfaces unresolved entries until they are repaired or trimmed

by the per-source per-role cap of 20 entries. Step logs are likewise trimmed to the most recent 24 appended sections. Critic, Inductive Explorer, Transductive Explorer, and Team Leader read the ledger as analytical input but do not own its entries.

B.2 Reflection and recovery flows

The per-step loop of Section B.1 runs unmodified for the common case in which the Team Leader commits to a non-RESET action and the environment returns a normal frame. Three flows wrap that loop and trigger when the harness detects a state that warrants reflection: reset reflection when a role proposes RESET, level reflection when a level completes, and crash recovery when an exception escapes a role's safety round. Each flow uses the same agent infrastructure as the step loop but with a different directive, a different context surface, and a different commit semantics.

Reset reflection. A RESET commitment is treated as a workspace event rather than a normal action. When the Team Leader proposes RESET in Phase 8, the harness intercepts the commitment before invoking $\mathrm{env.step}(\texttt{RESET})$ and runs a two-phase reflection. In phase one, all six roles read a *reset-reflection directive* that asks whether the proposal is justified, with a context surface that uses the recent action history of the current level rather than the new-level surface. The Team Leader emits one of three decisions: REJECT (no environment action; the proposal is withdrawn and play continues from the same scene), RESET (environment reset, workspace context preserved; we also call this a *soft RESET*), or RESET_FRESH (environment reset, strategy and observation context wiped). The same reflection runs unconditionally on a GAME_OVER recovery, with `harness_triggered = True` so that the REJECT option is masked and only the two real reset variants remain. A per-level cap on REJECT decisions blocks indefinite refusal: once the cap is reached on a level, REJECT is removed from the allowed set and the directive narrows to RESET versus RESET_FRESH. After a RESET executes, a cooldown of 5 environment actions blocks any subsequent RESET commitment, with the gate enforced both at directive-render time and at the pre-$\mathrm{env.step}$ assertion.

Level reflection. When the environment reports $\text{levels_completed}_{t+1} > \text{levels_completed}_t$, the harness pauses the step loop and runs a level-reflection round. All six roles activate concurrently, each receives a *level-reflection directive* and a context surface restricted to the completed level (no new-level pixels, no new-level metadata), and writes a reflection section to its `*_level_log.md`. No environment action is consumed. The harness checks afterwards that each role appended a non-trivial section, logs a warning when a role wrote less than 80 bytes, and clears the activation segment so that the next-level step loop starts as a fresh segment. Level logs accumulate across levels and survive RESET; they constitute the long-term memory that conditions later levels of the same game.

Crash recovery. A crash is any unhandled exception that escapes a role's safety round. The recovery contract has four rules. First, the harness logs a Python traceback and emits a structured `crash_recovered` event to every role's logger. Second, it picks a fallback action through a fixed priority chain:

1. the *preferred* action when one is available from an in-flight multi-action batch (the next planned action that has not yet executed);
2. `UNDO` when `UNDO` is in the available-actions set;
3. the most recent non-RESET action committed during the current run;
4. a step-zero scaffold of `USE`, `CLICK 32 32`, or `UP`, whichever is first available.

Third, the fallback is executed through the same single-action path as a normal commitment, so all bookkeeping (transition event, ledger update, step-log trim, agent-chat save) fires. Fourth, the harness sets a one-shot crash-recovery banner that is rendered at the top of the next step's primacy zone for every role, so the Team Leader sees what was tried and why on the next iteration. The chain refuses to emit RESET as a fallback under any circumstance; when no safe non-RESET option exists, the harness raises rather than silently committing to a state-erasing action.

B.3 Role scheduling and credit assignment

When each role acts. Six roles share the step. Each one has a fixed input surface and a fixed output surface defined by the file registry:

- Observer reads o_t, prior z_{encoded}, and reconstruction checks; writes `observable.py`, z_{encoded}, `level_constants`, and its step log.
- Simulator reads z_{encoded} (via the start-after gate), action history, prior h_{encoded}, and ledger entries with source `predict`; writes `dynamics.py` (which exposes `predict`, `history`, and `HYPOTHESES`), h_{encoded}, and its step log.
- Inductive Explorer reads z_{encoded} and h_{encoded} with diffs, retrodiction summaries, sub-goal acquisition state, and policy rollout results; writes `strategy.py` (which exposes `SUB_GOALS` and `POLICIES`) and its step log.
- Transductive Explorer reads grid history, the live `HYPOTHESES` table with per-hypothesis retrodiction accuracy, and teammate proposals; writes probe proposals, hypothesis tags, and its step log.
- Critic reads the union of teammate step logs, the data-quality report, the goal-acquisition view, and policy rollouts; writes critique notes, owner assignments, and its step log.
- Team Leader reads grid history, world-model status, goal acquisition, policy rollouts, teammate step logs, and asynchronous chat; writes the action commitment and its step log.

How handoffs occur. Handoffs are mediated by the shared file system and tool-gate dependencies. A role finishes when it writes its required outputs or exhausts its depth. Other roles read those outputs through tail-follow middleware that surfaces the most recent appended sections. Asynchronous chat is the one channel that does not flow through file appends; it carries short messages between roles within a step and is rendered at a fixed slot in each role's middle zone. The dependency graph is a directed acyclic graph rooted at the Observer and terminating at the Team Leader, with the Critic as a parallel evaluator that consumes from the four worker roles and is consumed by the Team Leader.

Credit assignment by ownership. A retrodiction failure carries an owner tag computed at evaluation time: `predict` failures go to the Simulator, `render` failures to the Observer, `POLICIES` or `SUB_GOALS` failures to the Inductive Explorer. The owner tag is a string written into the ledger entry, not a gradient. The patch that follows is bounded to the artifact owned by that role (`observable.py` for the Observer, `dynamics.py` for the Simulator, `strategy.py` for the Inductive Explorer), so the address of the repair is structural rather than statistical. No model parameters change during a run. The persistent training signal is the workspace itself, and credit assignment is the routing of a counterexample to a typed slot.

B.4 Repair feedback

Validation checks. Three checks run in sequence whenever a role writes an executable artifact. First, a Python source validator parses the new file and reports the syntax error if parsing fails. Second, the world-model loader registers the new module in `sys.modules`, replacing the previous version. Third, world-model validation reruns predict and render on the live transition window and reports per-field accuracy. The first check rejects unparseable code outright. The second and third checks are observational: they record the post-edit state and emit a feedback block, but they do not roll back the file. The role observes the consequence of its edit on the same step.

Regression replay. After a patch lands, the error-ledger middleware runs a fresh retrodiction summary across the recent action history for the current level. Each prior transition is replayed as

$$\hat{z}_s = \text{predict}(z_{s-1},\, h_s,\, a_{s-1},\, c_\ell,\, m_s),$$

and the diff against z_s is recorded per field. A ledger entry whose retrodiction now passes is marked `resolved_at = t`, and the resolution notice is queued to the owning role exactly once. A ledger entry that still fails remains unresolved with its original owner. The role sees the entry on the next step in the persistent ledger block, with its accumulated diff fields rather than only the most recent failure.

Tie-breaking and conflict. Two situations require a deterministic decision. First, when a single transition has both a parse mismatch on z_{t-1} and a prediction mismatch on z_t, the Observer is held responsible for the parse component and the Simulator for the prediction component, because the diff structure separates per-field encoding errors from per-field prediction errors. Second, when two roles

edit overlapping responsibilities (for example, the Inductive Explorer adding a progress predicate that the Simulator has not modeled yet), the Critic flags the conflict in its step log and the Team Leader either delays the commitment for one step or commits to a probe rather than a planned action. The schema-level disjointness defined in Appendix C prevents the two roles from writing the same field through normal patches.

Behavior after a failed repair. A failed repair produces three observable consequences. The ledger entry stays unresolved, so the owner sees the same routed counterexample on the next step. The world-model status block reports the post-edit accuracy at the field level, so the owner sees which fields are still wrong. The Critic, reading the ledger as input, can flag the patch as overfitting if it resolves the latest failure but breaks earlier transitions; in that case the entry for an earlier step transitions from resolved to unresolved and the Critic's note carries the regression delta. There is no automatic rollback. Repeated failures lengthen the entry's footprint on the persistent ledger, which is the signal the Team Leader uses to commit to a probe rather than a planned action.

Alternative role decompositions. The six-role decomposition is one cut through the labor of constructing a testable world model. Two alternatives appear in the literature and clarify what the cut buys. A single-agent decomposition collapses $\{\text{Obs}, \text{Sim}, \text{IE}, \text{TE}, \text{TL}, \text{Crit}\}$ into one function and folds the patch routing into a single inner monologue; the workspace is a single artifact and the Critic role disappears. A two-agent decomposition splits the team into a world modeler and a planner, leaving the rest implicit; this preserves the surface boundary between dynamics and strategy but loses ownership-level patch addresses for parsing, probing, and critique. The six-role cut chosen here has one role per typed surface, which is what makes the routing in Section B.1 address-stable: a contradicted commitment names a specific owner because each role owns exactly one executable artifact or one context surface.

C Artifact Schemas and Seed Workspace

This appendix gives the schemas and seed-workspace details deferred from Sections 2 and 4. The workspace components and the observation dataset map to concrete storage: $\mathcal{P}_t$ holds Python interfaces, $\mathcal{L}_t$ holds Markdown role logs, and $\mathcal{D}_t$ holds JSONL traces alongside the workspace; the harness draws $\mathcal{R}_t$ from the JSONL trace. Section 4 gives the role-by-role layout, and Appendix D covers invocation order, context assembly, and runtime mechanics.

C.1 Workspace schemas

The workspace is typed by four dataclasses. `ZState` stores the per-step structured observation, `HState` stores the dynamics hidden state propagated across steps, `LevelConstants` stores level-scoped invariants, and `Metadata` stores harness-provided context. The Observer owns `ZState` and `LevelConstants`; the Simulator owns `HState`; the harness writes `Metadata` and the agent reads it. Each is a flat dictionary at runtime, so the agent edits the dataclass declaration and the encoder writes a new JSON-compatible dictionary on the next step.

Observation state (`ZState`). `ZState` captures everything the Observer encodes from the current grid that may change between steps. The seed schema retains five fields and the Observer extends or prunes them as the game's vocabulary becomes clearer.

```
@dataclass
class ZState:
    object_positions: dict = field(default_factory=dict)
    object_states:    dict = field(default_factory=dict)
    sprite_overrides: dict = field(default_factory=dict)
    hud_values:       dict = field(default_factory=dict)
    event_objects:    dict | None = None
```

`object_positions` maps an entity name to its instance positions `[[row, col], ...]`. `object_states` stores per-entity mutable values (toggles, counters, modes). `sprite_overrides` records per-step deviations from `LevelConstants.objects`, keyed by `name` or `name:idx` with optional `rotation`, `pixels`, or `visible`. `hud_values` maps each HUD indicator name to its numeric

value. event_objects encodes a transient mini-animation triggered by the action (frame_count and a list of regions cross-referenced with entities), or None when no event occurred.

Dynamics hidden state (HState). HState stores only the recurrent state that history() needs to propagate forward. Long-lived dynamics knowledge that should survive a soft reset lives in code-level constants such as LEARNED_EFFECTS rather than in HState.

```
@dataclass
class HState:
    object_velocities: dict = field(default_factory=dict)
    event_triggers:    dict = field(default_factory=dict)
    game_state:        dict = field(default_factory=dict)
```

object_velocities maps an entity name to its step-wise velocity [dr, dc]. event_triggers accumulates {count, total_frames} per "ACTION:entity" pair. game_state is a free-form dictionary that the Simulator widens as the game's state machine becomes clear (for instance {"gravity_down": True, "selected_piece": "piece_0"}).

Level constants (LevelConstants). Per-level invariants live in LevelConstants. They are written to data/level_constants.jsonl as an append-only stream so the harness can rerender past levels for retrodiction.

```
@dataclass
class LevelConstants:
    background_color: int  = 0
    camera:           dict = field(default_factory=dict)
    walls:            list = field(default_factory=list)
    objects:          dict = field(default_factory=dict)
    hud:              list = field(default_factory=list)
```

background_color is the dominant background pixel. camera carries an optional viewport {vx, vy, vw, vh, padding_color} for upscaled or letterboxed levels. walls is a static 64×64 overlay with -1 marking transparent cells. objects maps each entity name to either a 2D pixel array or a richer record {pixels, layer, rotation}. hud is a list of indicator templates such as horizontal or vertical bars.

Metadata (Metadata). Metadata is harness-owned and read-only for the agent. It is updated before each call to history(), predict(), or policy().

```
@dataclass
class Metadata:
    step:              int  = 0
    level:             int  = 0
    available_actions: list = field(default_factory=list)
    grid_size:         int  = 64
```

C.2 Function signatures

Three artifacts expose a small callable interface. The harness loads a fixed set of named exports from each artifact: render and render_event from observable.py; predict, history, and HYPOTHESES from dynamics.py; SUB_GOALS and POLICIES from strategy.py. Table 4 summarizes the entry points.

Observation model. render reconstructs a 64×64 grid from the encoded state and the level invariants. render_event reconstructs the sampled transient frames associated with an event, returning an empty list when no event occurred.

```
def render(z_encoded: dict, constants: dict) -> list[list[int]]:
    """Reconstruct a 64x64 grid from (z_encoded, constants).
    Layers (back to front): background, walls, entities (sorted by
    layer, applying overrides and rotations), upscaling, HUD."""
```

Table 4: Harness entry points loaded into the agent's REPL. Each artifact declares a fixed named export set; new helpers are private to the artifact.

Artifact	Exports	Owner
`harness.py`	`Metadata`	Harness
`observable.py`	`render`, `render_event`	Observer
`dynamics.py`	`predict`, `history`, `HYPOTHESES`	Simulator
`strategy.py`	`SUB_GOALS`, `POLICIES`	Inductive Explorer

```
def render_event(z_encoded: dict, constants: dict) -> list[list[list[int]]]:
    """Reconstruct the sampled event frame sequence, or [] if no event."""
```

The pixel-accuracy loss compares `render(z_encoded, constants)` against the observed grid; the event-accuracy loss compares `render_event(z_encoded, constants)` against the observed event frames. Both losses surface to the Observer as named retrodiction errors.

Dynamics model. `predict` returns the next-step `ZState`-shaped dictionary from the previous state, the recurrent `HState`, and the action just taken. `history` returns the next `HState` from the previous `HState`, the previous observation, and the action.

```
def predict(z_prev: dict, h_encoded: dict, action_prev: str,
            constants: dict, metadata: dict,
            hypothesis: str | None = None) -> dict:
    """Project z_predicted in ZState shape from (z_prev, h_encoded,
    action_prev). Branch on `hypothesis` for uncertain mechanics; when
    omitted, the highest-probability entry of HYPOTHESES is used."""

def history(h_prev: dict, z_prev: dict, action_prev: str,
            constants: dict, metadata: dict) -> dict:
    """Propagate HState forward from (h_prev, z_prev, action_prev)."""

HYPOTHESES: dict[str, float]  # name -> probability, sums to 1.0
```

`HYPOTHESES` is a probability mass over named hypotheses about uncertain mechanics. The harness retrodicts every entry and uses per-hypothesis residuals to update the mass; the Simulator branches on `hypothesis` inside `predict` only for the parts of the dynamics it cannot yet commit to deterministically.

Strategy library. The Inductive Explorer maintains two named export tables. Each entry is a state-dependent callable that reads the workspace state and returns either an action (for policies) or a progress tuple (for sub-goals).

```
def policy_fn(z_encoded: dict, h_encoded: dict,
              constants: dict, metadata: dict) -> str: ...

def sub_goal_fn(z_encoded: dict, h_encoded: dict,
                constants: dict, metadata: dict
               ) -> tuple[bool, float, str]:  # (achieved, progress, reason)
    ...

POLICIES:  dict[str, callable]  # name -> policy_fn
SUB_GOALS: dict[str, callable]  # name -> sub_goal_fn
```

Policies that ignore workspace state are logged as probes in the Transductive Explorer's step log rather than registered in `POLICIES`. The sub-goal triple returns whether the goal is satisfied, a progress fraction in $[0, 1]$, and a short `reason` string the Critic and Team Leader read.

Rollout primitive. The rollout used in Algorithm 1 is a harness composition: given (z_t, h_t) and a policy π, it iterates $a_{t+i} = \pi(z_{t+i}, h_{t+i}, \dots)$, $h_{t+i+1} = \texttt{history}(h_{t+i}, z_{t+i}, a_{t+i}, \dots)$, and

$\hat{z}_{t+i+1} = \texttt{predict}(z_{t+i}, h_{t+i+1}, a_{t+i}, \dots)$ for $i < k$, scoring the trajectory against `SUB_GOALS` at each step. Rollout-readiness is the conjunction of well-defined `predict`, `history`, and the relevant `POLICIES` and `SUB_GOALS` entries.

C.3 Seed workspace artifacts

When a game starts, the harness deploys a fixed set of files into the team's shared directory. Existing files are preserved across resets so that learned content survives. The deployment list is summarized in Table 5.

Table 5: Seed file inventory. The harness installs these files at game start; existing files are preserved across soft resets so that learned schemas, dynamics, and strategy survive.

Path	Purpose	Owner
`world_model/harness.py`	`Metadata` dataclass and shared utilities	Harness
`world_model/observable.py`	`ZState`, `LevelConstants`, `render`, `render_event`	Observer
`world_model/dynamics.py`	`HState`, `predict`, `history`, `HYPOTHESES`, `LEARNED_EFFECTS`	Simulator
`world_model/strategy.py`	`SUB_GOALS`, `POLICIES` tables	Inductive Explorer
`data/z_encoded.jsonl`	Per-step `ZState` trace	Observer
`data/h_encoded.jsonl`	Per-step `HState` trace	Simulator
`data/level_constants.jsonl`	Append-only `LevelConstants` stream	Observer
`*_step_log.md`	Per-role per-step working log	Each role
`*_level_log.md`	Per-role end-of-level summary	Each role
`*_helper.py`	Per-role REPL helpers, auto-loaded each step	Each role

Seed variants. Two seed variants are available: a *simple* seed and a *rich* seed. The simple seed retains `ZState` fields `object_positions`, `object_states`, and `event_objects`, and `LevelConstants` fields `background_color`, `walls`, and `objects`. The rich seed adds `sprite_overrides` and `hud_values` to `ZState` and `camera` and `hud` to `LevelConstants`; its dynamics seed adds `game_state` to `HState` and broader `LEARNED_EFFECTS` fields including rotation deltas and visibility. Variant selection is per-game, controlled by the league configuration. The simple seed encourages a conservative encoding that the Observer extends only when forced by a parse failure; the rich seed biases toward upscaled and HUD-heavy games where overlays appear in the first few frames.

Seed defaults. The seeded `render` stamps the background color, applies the wall overlay where pixels are nonnegative, then stamps each entity's sprite at every position from `object_positions`, with `sprite_overrides` replacing the base pixels when present. The seeded `predict` carries each entity forward by the action's effect from `LEARNED_EFFECTS` when known, otherwise by the entity's velocity from `HState` when known, otherwise leaves the position unchanged. The seeded `history` carries forward observed velocities and accumulates event-trigger statistics from `z_prev["event_objects"]`. The seeded `HYPOTHESES` contains a single primary entry; the Simulator widens it once retrodiction surfaces a mechanic it cannot commit to deterministically. The seeded `SUB_GOALS` and `POLICIES` tables are empty and the Inductive Explorer fills them as rollout-supported plans accumulate.

Code-level constants. The dynamics artifact also exposes `LEARNED_EFFECTS`, a code-level dictionary that stores confirmed action-to-entity effects discovered through retrodiction. Its schema is

```
LEARNED_EFFECTS: dict[str, dict[str, dict]]
# {ACTION: {entity_name: {dr: int, dc: int, drot: int, visible: bool}}}
```

where the inner record carries the row delta, column delta, rotation delta, and visibility change applied to each instance of `entity_name` when the player commits `ACTION`. Storage at the code level (rather than inside `HState`) is what lets a confirmed effect survive a soft reset that wipes the runtime trace.

Helper files. Each role has a persistent `*_helper.py` file that the harness auto-loads into the role's REPL at the start of every step. Helpers are private to a role and do not export through

HARNESS_FUNCTIONS; they exist so a role can build internal utilities (entity-matching predicates, rollout scoring helpers, debugging probes) without cluttering the four exported artifacts. Helpers are part of the workspace in the sense of Section 4: edits accumulate across steps, the Critic can read them, and they are subject to the same accept-on-evidence discipline as the executable artifacts.

C.4 Schema disjointness

The workspace is partitioned so that each piece of evidence is owned by exactly one artifact. Disjointness gives the harness a deterministic routing rule: a counterexample component lands at the artifact whose schema covers the violated field. Table 6 lists each field and its owner.

Table 6: Field ownership across the workspace. Each field appears in exactly one schema and is written by exactly one role; retrodiction errors over a field are routed to that role.

Schema	Field	Owner	Repair signal
ZState	object_positions	Observer	Position parse error
ZState	object_states	Observer	State decoding error
ZState	sprite_overrides	Observer	Per-step appearance error
ZState	hud_values	Observer	HUD readout error
ZState	event_objects	Observer	Event encoding error
HState	object_velocities	Simulator	Velocity prediction error
HState	event_triggers	Simulator	Trigger statistic error
HState	game_state	Simulator	State-machine error
LevelConstants	all fields	Observer	Pixel-accuracy regression
Metadata	all fields	Harness	(read-only)

Forbidden overlaps. A field belongs to exactly one schema. The Observer writes ZState and LevelConstants but reads HState; the Simulator writes HState but reads ZState. A patch that moves an Observer-owned field into HState (for example, copying object_positions into the dynamics state) is rejected because it duplicates the source of truth and lets the two schemas drift. The same constraint excludes Simulator-only state from ZState: velocities, action effects, and state-machine variables remain in HState or in code-level constants such as LEARNED_EFFECTS.

Tolerated overlaps. Two narrow overlaps are tolerated. First, entity *names* are shared across ZState.object_positions, HState.object_velocities, and LevelConstants.objects. The same canonical name keys all three so that a position, a velocity, and a sprite refer to the same entity. Second, event_objects appears in ZState as a per-step transient and event_triggers appears in HState as an aggregate statistic over those transients. The Observer encodes the per-step event; the Simulator accumulates the statistic. The two are tied by entity name and action, and either can be edited without touching the other.

Disjointness and localized repair. Disjointness makes the diff $e_t = \mathrm{diff}(\hat{z}_{t+1}, z_{t+1})$ routable. A field-level mismatch on ZState is owned by the Observer when the encoding of z_{t+1} disagrees with the new frame, or by the Simulator when the encoding agrees but predict did not project to it; a sub-goal disagreement under an agreed ZState is owned by the Inductive Explorer. The Critic intervenes only when more than one candidate fits a single failure.

C.5 Regression checks and retained evidence

The regression set $\mathcal{R}_t$ referenced in Sections 2 and 4 is not a separate test database; it is a window of past transitions that the harness rebuilds on every step by replaying the current programs against the stored trace. This appendix gives its data shape; Appendix D describes how the window is selected and how individual replays are orchestrated.

Per-step retrodiction record. Each entry in $\mathcal{R}_t$ is a structured record produced by replaying the current predict, render, and render_event on a stored transition $(z_{k-1}, h_k, a_{k-1}, z_k)$ for some $k \leq t$. The record is shaped as follows.

```
@dataclass
```

```
class StepRetrodiction:
    step:   int
    level:  int
    action: str
    source: str  # "recent" | "event" | "error_z" | "error_render"

    # predict(z[k-1], h[k], action, ...) vs z[k]
    z_predicted:  dict | None
    z_actual:     dict | None
    z_accuracy:   str | None        # "N/M" matched fields
    z_mismatches: list | None       # per-field counterexamples

    # render(z[k], constants) vs actual_grid[k]
    render_z_accuracy: float | None
    render_z_metrics:  dict | None  # {pixel_accuracy,
                                    #  structural_similarity,
                                    #  object_iou,
                                    #  color_histogram_distance}

    # render(predict(z[k-1], h[k], action, ...), constants) vs actual_grid[k]
    render_pred_accuracy:      float | None
    render_pred_by_hypothesis: dict   # name -> accuracy

    # render_event(z[k], constants) vs actual event frames at step k
    event_accuracy:  float | None
    event_agreement: str | None      # "match" | "missed"
                                      # | "false_positive" | "agree_none"

    # per-hypothesis retrodiction for HYPOTHESES table
    hypothesis_results: dict | None  # name -> {accuracy, mismatches,
                                      #         probability}

    # delta vs previous retrodiction of the same step
    improved:          bool | None
    improvement_detail: str | None
```

Window composition. The harness composes $\mathcal{R}_t$ from four selectors with fixed quotas: *recent* (last few transitions), *event* (transitions with events), *error_z* (worst per-field prediction errors), and *error_render* (worst pixel-accuracy errors). The pools are unioned and deduplicated, then rebuilt against the patched programs on every step, so a patch that improves a recent entry but breaks an older one surfaces immediately.

Accuracy metrics. Three families of metric appear in a record. Field-level accuracy compares `ZState` dictionaries entry by entry, yielding a matched-fields fraction and named mismatches. Pixel-level accuracy compares two 64×64 grids on four scores: raw pixel agreement, structural similarity, object-level IoU after instance matching, and color histogram distance. Event-level accuracy yields a categorical agreement label and a per-frame pixel score. The Observer reads pixel and event metrics, the Simulator reads field and per-hypothesis metrics, and the Critic reads the improvement delta.

Replay feedback against $\mathcal{R}_t$. Patches enter the workspace once their source passes a syntactic and exports check (the harness validates that the file loads and that its named exports are present and callable). There is no automatic acceptance gate against $\mathcal{R}_t$ and no rollback. The replay produces a per-record delta that the harness reports as feedback; records that the patch breaks become new counterexamples for the same owner on the next step. Repeated failures lengthen the entry's footprint on the persistent ledger, which is the signal the Team Leader uses to switch from planning to probing.

C.6 DREAMTEAM agentic workspace details

The trainable object in DREAMTEAM is the workspace, not only the executable world model. The world model is the most visible part of the workspace because it is runnable and regression-tested, but the agent also trains persistent role context around that code: logs, hypotheses, goals, probes,

critique notes, routing decisions, action commitments, strategy notes, and retained regression checks. A future model call therefore reads both programs and the structured evidence that explains why those programs have their current form.

We separate the workspace into executable artifacts and role-specific context.

Observation/world model. The observation artifact, implemented as `observable.py`, defines the structured observation schema and the renderers used to check a parse, not a reusable raw-grid parser. It contains `ZState`, entity labels, rendering helpers, and optional feature extractors. The Observer reads the latest screen, previous parsed state, parse failures, and reconstruction checks, then emits a per-step z_t record transductively. It may update entity definitions, schema fields, render helpers, and notes about ambiguous pixels or sprites. For example, after an unexpected reset caused by touching a newly seen object, the Observer may add an `obstacle` or `hazard` entity with a shape signature and a reconstruction test over the observed frame. The update enters the workspace once the artifact loads; replay and render checks then report whether it explains the current observation and whether older retained frames now regress. Any regression is surfaced as a counterexample for the next Observer update rather than causing an automatic rollback.

Dynamics/simulator. The dynamics artifact, implemented as `dynamics.py`, defines the predicted effect of actions on structured state. The Simulator reads `z_prev`, the committed action, the observed `z_next`, stored one-step predictions, and dynamics failure reports. It writes transition rules, constants, preconditions, and rollout checks. For example, a failed prediction that expected ordinary movement but observed death can become a lethal-contact rule guarded by the newly named entity type. Once the patch loads, the harness reports whether it fixes the failed transition and what happens on the retained regression window over previous movement, wall collision, button, reset, and inventory transitions. If the rule is plausible but underdetermined, it remains in the simulator notes as a candidate rule and the Transductive Explorer may be asked for a targeted probe.

Strategy/inductive explorer. The strategy artifact, implemented as `strategy.py`, defines reusable policies, subgoals, and progress checks. The Inductive Explorer reads the current observation state, simulator rollouts, known goals, failed tactics, and the Policies $\times$ Sub-Goals matrix. It writes named policies, goal predicates, rollout summaries, strategy notes, and regression checks for policy progress. For example, once the simulator can predict movement around a lethal object, the Inductive Explorer may add a `route_around_hazard` policy with a progress check requiring reduced distance to a target without entering lethal contact. Simulated rollouts and recent real evidence determine whether the policy is promoted into the reusable strategy library; if it succeeds only in the current trace, it may be retained as a local plan instead.

The non-executable role context is also part of the trainable workspace. It changes what later calls attend to, which actions are considered informative, and which failures are routed to code owners.

Transductive Explorer context. The Transductive Explorer owns local experimentation rather than a durable code artifact. It reads uncertainty tags, failed predictions, unexplained observations, the action budget, and the Team Leader's recent commitments. It writes probe proposals, hypotheses tagged `[UNTESTED]`, `[UNCERTAIN]`, or `[CONFIRMED]`, expected probe outcomes, and stop criteria. A concrete write might be: "Press the blue tile once; if the door opens, route to dynamics as a toggle rule; if only the sprite changes, route to observation; if neither changes, mark the tile non-interactive." Probe outcomes are accepted into the retained workspace when the environment result distinguishes hypotheses or creates a counterexample for an owner. Probes that do not disambiguate are kept as negative evidence so they are not repeated without a new reason.

Critic context. The Critic owns failure localization and quality control. It reads role logs, committed predictions, observed outcomes, proposed patches, action plans, and regression deltas. It writes critique notes, failure routing criteria, owner assignments, and blocking objections for the Team Leader and artifact owners. For example, when a patch adds a broad "all colored blocks are lethal" rule, the Critic may point to an earlier safe colored block and argue that the next commitment should be delayed until the rule is narrowed. Its updates are retained as routing constraints and regression targets: a note that a failure is parser-owned,

simulator-owned, or strategy-owned changes which artifact receives the next patch request. A Critic objection is authoritative only when it is grounded in a logged trace, violated commitment, or failed check; otherwise it remains as a warning rather than a blocker.

Team Leader context. The Team Leader owns action commitment and arbitration. It reads the Observer's parse, Simulator predictions, Inductive Explorer rollout results, Transductive Explorer probes, Critic warnings, action budget, and recent success or failure signals. It writes the selected action, the reason for the action, the prediction it is committing the workspace to, and the decision source: probe, rollout-supported plan, reusable policy, repair action, or fallback. For example, it may commit to `RIGHT` with the stored prediction that the avatar moves one cell right and no reset occurs. After the environment responds, that commitment becomes training evidence. If the prediction holds, the commitment is retained as support for the current parser and simulator; if it fails, the mismatch becomes a routed counterexample rather than an unstructured mistake.

Inductive Explorer context. In addition to owning `strategy.py`, the Inductive Explorer maintains notes that are not yet executable strategy. It reads Team Leader decisions, real progress signals, simulator rollout disagreement, and Critic warnings about degenerate policies. It writes candidate goals, abandoned tactics, rollout caveats, and promotion criteria for turning a local plan into a reusable policy. A concrete retained note might say that a key-collection route worked only after a door-toggle rule was added, so future policy reuse must require the door state to be known. Such notes are accepted when they summarize logged rollouts or real transitions, and they remain non-executable until converted into a policy or regression check.

Durability is deliberately conservative. Executable patches enter W_{t+1} once the harness confirms they load and that their named exports are present, and the harness then reports their replay fitness against the current counterexample and against $\mathcal{R}_t$ as per-record feedback. Non-executable context is retained when it is tied to a trace, a prediction, a probe outcome, a routing decision, or an action commitment. Unsupported claims can still remain in the workspace, but only with an uncertainty tag and without the authority to block a patch or justify a plan. In this sense the workspace learns at two levels: programs change the behavior of parsing, simulation, and policy rollout, while role context changes what evidence is collected, how failures are assigned, and which executable edits the role is willing to commit to next.

D Execution Details

This appendix contains implementation details deferred from Sections 4 and 5. Section D.1 describes how the Team Leader's per-step commitment is executed when it spans more than one environment action.

D.1 Multi-Action Execution Modes

The deliberation step described in Algorithm 1 commits a single action a_t. In practice, the Team Leader can commit a short batch of actions in one response. The harness then executes the batch under a configurable mode that controls how much intermediate computation runs between consecutive actions. The configuration ceiling is `max_actions_per_step`, with a default of twenty, and the Team Leader's response is parsed by extracting the action sequence from the first non-empty line of its structured output.

Three execution modes are supported. They differ in how the world model and the Team Leader are updated between actions, and in how early-stop triggers interrupt the batch.

Simple mode. Every action in the batch is executed against the environment without any intermediate language-model call. The Observer and Simulator do not run between actions; the harness collects observations $o_{t+1}, \ldots, o_{t+n}$, appends them to the trajectory, and lets the next deliberation step process the entire batch retroactively. Simple mode is selected when the world model holds high retrodiction accuracy on recent transitions and the action sequence consists of repeated movement in a region the parser already covers. Its purpose is throughput: a confident traversal of a known corridor avoids paying the language-model cost of a per-step deliberation.

Approval-based mode. After each action, the Observer and Simulator run lightweight micro-tasks: the Observer encodes the new screen into z_{t+i+1}, and the Simulator updates h_{t+i+1} and runs retrodiction on the just-observed transition. The Team Leader then reads the freshly computed state and chooses one of three outcomes for the next action in the batch: *approve* the next action and continue, *reject* the remainder of the batch and return to a full deliberation step, or *request a reset*. Approval-based mode lets the Team Leader stop a planned sequence at the first sign that the world model's prediction has drifted, while still amortizing the cost of the upstream Inductive Explorer rollout that produced the batch.

Policy-driven mode. The Team Leader commits to a registered policy and a length, written as `POLICY:<name>:<n>`. The harness invokes the policy on $(z_{t+i}, h_{t+i}, \dots)$ to choose action a_{t+i} for $i = 0, \dots, n-1$, runs Observer and Simulator micro-tasks after each action so the policy reads up-to-date state, and stops if the policy fails to produce a valid action. The Team Leader is consulted periodically rather than between every action; this lets a state-dependent policy from `strategy.py` carry the trajectory through a long routine while keeping the world model grounded.

z_{prev} chaining. In every mode that runs micro-tasks between actions, the Observer's encoding from the just-executed action is the z_{prev} for the next action's prediction and rollout. The chain is propagated through the harness rather than reconstructed from scratch, so a stale z_{prev} from before the batch started cannot leak into a later prediction. This makes multi-action retrodiction equivalent, action by action, to the single-action retrodiction described in Section 4.2: each action in the batch produces its own diff e_{t+i} that is routed to the same owners by the same disjointness rule.

Early-stop triggers. A batch in approval-based or policy-driven mode is interrupted by any of the following conditions, evaluated after each action: a level-completion increment in the harness's `levels_completed` counter; a non-empty transient event captured by the Observer's `event_objects` encoding; a `GAME_OVER` return from the environment; or a reset request raised by the Team Leader's approval gate. Each of these conditions either changes the workspace in a way that invalidates the precomputed batch (a level transition introduces new `LevelConstants`; an unexpected event surfaces a counterexample to the dynamics model) or signals that the agent should re-deliberate.

Crash recovery and the preferred action. The harness records the proposed batch at the start of execution. When an action mid-way through the batch raises an exception or times out, the next deliberation step starts with a *preferred action* hint set to the first unplayed action in the batch. The next Team Leader decision is biased toward continuing the planned sequence rather than restarting from scratch, so a transient failure in the middle of an approved trajectory does not cost the rest of the trajectory.

Action accounting. Multi-action is the only mechanism that produces more than one environment transition per deliberation step, and is therefore the dominant lever on the per-action cost reported in Section 5. The action budget against the human baseline counts environment transitions, so multi-action changes the language-model cost charged to each transition without changing the denominator.

E Additional Experiments

This appendix records the experimental setup for the runs analysed in Section 5, the per-run reproducibility breakdown, and the benchmark-version caveat needed to interpret comparisons against published ARC-AGI-3 numbers. Ablations, synthetic-game results, and game-family stratification are not claimed here.

E.1 Experimental setup

Table 7 records the per-role model assignment and the team-wide hyperparameters of the run analysed in Section 5.

Table 7: Experimental setup. DREAMTEAM on ARC-AGI-3 public set, competition mode, single seed.

Parameter	Value
Operation mode	competition
Games	25 (public set)
Seed	0
Dedicated step-wall cap	none in the analysed run
Per-agent deliberation cap	300 s
LLM-call timeout	180 s
Team-leader timeout	300 s
Max actions per step (MA)	20
Reset cooldown	5 steps
Safety-round depth	4
Safety-round breadth	6
Per-step token budget	250,000 tokens
Context window (last-k)	6 rounds, $\leq$ 500K tokens
Reasoning compaction	strip after 1 turn
Summarization	on (FLASH, $\leq$ 50K tokens)
Policy-action execution	enabled
Reset reflection	enabled
DSL injection in prompts	off
Inter-role chat channel	off (file-based comms only)
Per-role models	
Observer / Simulator	Claude Opus 4.6 (no reasoning)
Team Leader / TE	Claude Opus 4.6 (low reasoning)
Inductive Explorer / Critic	GPT-5.5 (high reasoning)

E.2 Run-to-run reproducibility

The headline numbers in Section 5 average over two independent runs of DREAMTEAM on the same 25 games with identical hyperparameters (Table 7). Run 1 completed all 25 games within the protocol budget; run 2 was operator-stopped while 18 games were still in progress. For those 18 games we read the *partial-run RHAE* from per-step `level_score` events through the run-2 stop point; the formula is the same as the official scorer (Eq. 1–2). Table 8 shows the pairwise per-game RHAE on the seven games that completed in both runs.

Table 8: Per-game RHAE (%) on the seven games that completed in both runs. Mean delta across the seven (run 2 − run 1): -0.20 pp, median 0. Notable per-game movements: R11L (run 2 cleared L4–L5 that run 1 missed) and SB26 (run 2 regressed on later levels) cancel each other out.

Game	Run 1	Run 2	Per-game mean
AR25	100.00	100.00	100.00
CD82	79.05	90.41	84.73
FT09	100.00	92.24	96.12
LP85	100.00	100.00	100.00
R11L	47.62	88.62	68.12
SB26	93.96	47.97	70.97
TU93	100.00	100.00	100.00

Aggregated across all 25 games, run 1 alone reaches mean RHAE 38.06% on $9{,}789$ env actions and run 2 reaches 38.67% on $12{,}391$ env actions (the latter using partial-run RHAE for the 18 games not yet finalised when run 2 was stopped). The per-game mean of the two runs is $\mathbf{38.36}\%$ at a mean of $\mathbf{444}$ env actions per game; this is the number reported in Section 5. The per-step pooled denominator ($22{,}180$ env actions across the two runs) is what Table 3 uses for per-role averages.

E.3 RHAE scoring formula

The official ARC-AGI-3 scorer[4] computes RHAE in three stages. Let $B_{g,\ell}$ be the human-baseline action count for game g, level ℓ, and $a_{g,\ell}$ the agent's action count on that level. The per-level score is

$$\mathrm{RHAE}_\ell(g) \;=\; \min\left(1.15,\; \left(\frac{B_{g,\ell}}{a_{g,\ell}}\right)^2\right), \tag{1}$$

i.e., the squared action ratio capped at 1.15. The per-game score is the level-weighted mean using level-index weights $w_\ell = \ell$ (1-indexed), bounded by the fraction of weights for levels actually solved:

$$\mathrm{RHAE}(g) \;=\; \frac{\sum_{\ell \in \mathrm{solved}(g)} \ell \cdot \mathrm{RHAE}_\ell(g)}{\sum_{\ell=1}^{L_g} \ell}, \tag{2}$$

where L_g is the total number of levels in game g. Unsolved levels contribute zero to the numerator while still counting in the denominator, so completion is a hard ceiling: a game cannot reach 100% without all levels cleared. The headline number is the unweighted mean of $\mathrm{RHAE}(g)$ over the 25 public games.

E.4 Benchmark version shift and human-baseline drift

The ARC-AGI-3 changelog for April 14, 2026 changed the public scoring methodology and republished fifteen game versions.[5] The scoring change is itself important: the human baseline now uses the median human per level rather than the second-best human, and the per-level score cap increased from $1.0\times$ to $1.15\times$ the human baseline. Comparisons across this boundary therefore conflate agent quality, benchmark content, and scoring protocol. We consequently pin every experiment to exact game hashes and treat leaderboard numbers from the older versions as contextual rather than protocol-matched evidence.

To quantify the direction of the content shift, let $B_{g,\ell}$ denote the published human-baseline action count for game g and level ℓ, and let $H_g = \sum_\ell B_{g,\ell}$. We use H_g as a benchmark-native proxy for action-efficiency pressure: larger values indicate a longer median-human solution and therefore a larger action budget under normalized scoring. This is not a semantic difficulty metric. A game can become easier for humans but harder for an agent if the changed mechanic stresses perception, memory, or exploration. It is nevertheless the budget signal used by the benchmark and by our reset thresholds.

Across the fifteen republished games, the aggregate baseline increased from $9{,}215$ to $10{,}253$ actions, a net increase of $1{,}038$ actions or 11.3% when weighted by old baseline size. Averaging the per-game relative changes gives a larger 15.8% increase, because several smaller games were revised upward substantially. Eleven of the fifteen games became harder by more than two percent; restricted to those games, the old and new totals are $6{,}597$ and $7{,}926$, a 20.1% weighted increase and a 25.7% mean per-game increase. Two games became easier, and two are effectively unchanged by total baseline actions.

Table 9: Aggregate shift in human-baseline action budgets for the fifteen ARC-AGI-3 games republished on April 14, 2026. Weighted relative change is computed from total old and new baseline actions; mean per-game change averages game-level percentage changes.

Subset	Games	Old total	New total	Weighted rel.	Mean rel.
All republished games	15	9,215	10,253	+11.3%	+15.8%
Harder games only	11	6,597	7,926	+20.1%	+25.7%
Easier games only	2	1,392	1,091	-21.6%	-24.0%
Near-ties	2	1,226	1,236	+0.8%	+0.6%

Table 10 shows that the largest upward revisions are SC25, SK48, R11L, AR25, and TN36. KA59 and SU15 became easier by this proxy. VC33 is unchanged, and CN04 is a near-tie by total baseline actions despite a substantial content change: it moves from five to six levels, adds per-level step

[4] `https://docs.arcprize.org/methodology`, accessed May 7, 2026.
[5] `https://docs.arcprize.org/changelog`, accessed May 5, 2026.

Table 10: ARC-AGI-3 version shift on April 14, 2026, measured by total human baseline actions across levels. Larger totals indicate a longer median-human solution and are treated as harder for budget-normalized evaluation. A two-percent deadband is labeled near-tie.

Game	Old total	New total	Delta	Rel.	Direction
SC25	216	350	+134	+62%	Harder
SK48	696	1070	+374	+54%	Harder
R11L	167	233	+66	+40%	Harder
AR25	577	748	+171	+30%	Harder
TN36	250	317	+67	+27%	Harder
TU93	378	462	+84	+22%	Harder
RE86	1071	1255	+184	+17%	Harder
M0R0	970	1107	+137	+14%	Harder
SP80	472	518	+46	+10%	Harder
S5I5	608	638	+30	+5%	Harder
DC22	1192	1228	+36	+3%	Harder
CN04	779	789	+10	+1%	Near-tie
VC33	447	447	0	0%	Near-tie
KA59	826	730	-96	-12%	Easier
SU15	566	361	-205	-36%	Easier

limits, and changes later-level masking/background structure. These mixed cases are the main reason we report exact game hashes rather than only game names.